



GEOSPARQL 3D WHITE PAPER

TECHNICAL PAPER

DRAFT

Version: 1.0

Submission Date: 2029-03-30

Approval Date: 2029-03-30

Publication Date: 2029-03-30

Editor: Editor One, Editor Two

Notice: This document is not an OGC Standard. This document is an OGC Technical Paper and is therefore not an official position of the OGC membership. It is distributed for review and comment. It is subject to change without notice and may not be referred to as an OGC Standard. Further, an OGC Technical Paper should not be referenced as required or mandatory technology in procurements.

License Agreement

Use of this document is subject to the license agreement at <https://www.ogc.org/license>

Copyright notice

Copyright © 2026 Open Geospatial Consortium

To obtain additional rights of use, visit <https://www.ogc.org/legal>

Note

Attention is drawn to the possibility that some of the elements of this document may be the subject of patent rights. The Open Geospatial Consortium shall not be held responsible for identifying any or all such patent rights.

Recipients of this document are requested to submit, with their comments, notification of any relevant patent claims or other intellectual property rights of which they may be aware that might be infringed by any implementation of the standard set forth in this document, and to provide supporting documentation.

CONTENTS

I. KEYWORDS	vii
II. PREFACE	viii
III. SECURITY CONSIDERATIONS	ix
IV. SUBMITTING ORGANIZATIONS	x
V. SUBMITTERS	x
1. SCOPE	2
2. CONFORMANCE	4
3. NORMATIVE REFERENCES	6
4. TERMS AND DEFINITIONS	8
5. ABSTRACT	10
6. KEYWORDS	12
7. CONVENTIONS	14
8. INTRODUCTION	16
9. MOTIVATION	18
9.1. Why do we want 3D semantic modeling?	18
9.1.1. A geo-spatial 3D specification would support data consumers by:	19
9.1.2. A geo-spatial 3D specification would support data providers by:	19
10. RELATED WORK	21
10.1. CityGML	21
10.1.1. CityJSON and CityRDF	21
10.2. Medicine & Chemistry	22
10.3. 3D data formats and ontologies	23
10.3.1. STereoLithography (STL)	23
10.3.2. Wavefront OBJ Format (OBJ)	23
10.3.3. Polygon File Format (PLY)	23
10.3.4. CARARE Metadata Schema	24
10.3.5. IIIF 3D	24

10.3.6. X3D and X3D Ontology	24
10.3.7. Geometry Metadata Ontology (GOM)	24
10.3.8. Ontology for Managing Geometry (OMG)	24
10.3.9. File Ontology for Geometry Formats (FOG)	25
10.4. To consider	25
10.5. Cultural Heritage	25
10.5.1. Use Cases	26
10.5.2. Research applications making use of 3D models in Cultural Heritage	26
10.5.2.1. 3D models of cuneiform tablets	26
10.6. IFC and BIM	27
10.6.1. Industry Foundation Classes (IFC) and BIM	27
10.6.1.1. Product model	27
10.6.1.2. Evolving views on geometry	28
10.6.1.3. Use cases	29
10.6.1.4. Implications and questions:	29
10.7. Implementations	30
10.7.1. CGAL → SFCGAL → PostGIS	30
10.7.2. OpenCASCADE	30
10.7.2.1. OpenCASCADE-inspired BRep ontology	30
10.7.2.2. Topologic	30
10.7.3. BRep vs mesh/polyhedron	30
11. CURRENT CAPABILITIES	33
11.1. GeoSPARQL	33
11.1.1. Requirements addressed	33
11.1.1.1. Dimensionality	33
11.1.1.2. Vocabulary	34
11.1.1.3. Core Module	34
11.1.1.4. Query functions with 3D support	37
11.1.2. Adoption of GeoSPARQL 1.1	37
11.1.2.1. GeoSPARQL Alignments	37
11.1.2.2. Geometry Topology Extension	37
11.1.2.3. RDFS Entailment Extension	37
11.1.2.4. Query Rewrite Extension	38
12. REQUIREMENTS FOR GEOSPARQL 3D	40
12.1. 5.1 Existing implementation of 3D geometry in GeoSPARQL	40
12.1.1. Proposed extensions for GeoSPARQL 3D	40
12.1.1.1. Extension 1: 3D representations	40
12.1.1.2. Extension 2: Topological relations of 3D geometries	41
12.1.1.3. Extension 3: Directional relations of 3D geometries	44
12.1.2. Proposed extensions for GeoSPARQL 3D	45
12.1.2.1. Extension 4: Routing	45
12.1.2.2. Extension 5: Appearance of 3D geometries	45
12.1.2.3. Extension 6: Multi-component 3D geometries	46
12.1.2.4. Extension 7: Positioning of 3D geometries	46
12.1.2.5. Extension 8: Alignments of GeoSPARQL 3D	47

12.1.2.6. Extension 9: Alignments of Engineering CRS to Geospatial CRS	48
12.1.2.7. Extension 10: Geometry Extrusion	48
12.1.2.8. Extension 11: Geometry Attributes	49
12.1.2.9. Extension 12: Non-topological Query Functions – 3D Extension	49
12.2. 5.2 3D Geometry available in IFC	50
12.3. Methodology for testing prevalence.	57
12.4. 5.3 Vanilla 3D geometry handling in the Semantic Web	59
12.5. 5.4 Concluding overview of requirements for 3D geometry in the semantic web	59
13. BENEFICIARIES AND BENEFITS	61
13.1. Semantic-functional value of 3D-GeoSPARQL	61
13.1.1. Meaningful creation and querying of 3D geometry	61
13.1.2. Meaningful creation and querying of 3D topological relationships	61
13.1.3. Derivation of geometric and topological knowledge	62
13.2. Value for user experience, visualization, and application use of 3D-GeoSPARQL	62
13.2.1. Visualization	62
13.2.2. Calculation and analysis	62
13.2.3. Spatial querying	62
13.2.4. Machine control	63
13.2.5. Modeling, simulation and planning	63
13.3. Added value of the 3D-GeoSPARQL for Organisations	63
13.3.1. Brings together different domains (GIS, BIM, CG)	63
13.3.2. Enables extension to vocabularies from other domains	63
13.3.3. Facilitates better interoperability between knowledge domains with spatial components (geography, aerospace, architecture, product design, (bio)chemistry, IoT, ...)	63
14. ANNEX N: ALIGNMENT GEOSPARQL AND OPEN SOURCE RELATIONAL DATABASES WITH GEOSPATIAL SUPPORT	65
15. ALIGNMENT: GEOSPARQL AND OPEN SOURCE RELATIONAL DATABASES WITH GEOSPATIAL SUPPORT	67
15.1. Formatting functions	67
15.2. Convert geometries	68
15.3. Create geometries	70
15.4. Edit geometries	77
15.5. Topological Relations	85
15.6. Measure	89
15.7. Count	93
15.8. Properties	93
15.9. Topography	98
15.10. Triangulation	99
15.11. Export/export	100
15.12. Other	101
16. ANNEX B: ALIGNMENT OF GEOMETRY	110
17. ANNEX O: HISTORY	127

BIBLIOGRAPHY	131
--------------------	-----



KEYWORDS

The following are keywords to be used by search engines and document catalogues.

OGC, GeoSPARQL, 3D



PREFACE

To come...



SECURITY CONSIDERATIONS

The following security considerations apply...

IV

SUBMITTING ORGANIZATIONS

The following organizations submitted this Document to the Open Geospatial Consortium (OGC):

- Organization one
- Organization two
- Organization three

V

SUBMITTERS

NAME	AFFILIATION	OGC MEMBER
Nicholas J. Car	KurrawongAI	Yes



1 SCOPE



SCOPE



2

CONFORMANCE



CONFORMANCE



3

NORMATIVE REFERENCES



NORMATIVE REFERENCES

There are no normative references in this document.



4

TERMS AND DEFINITIONS



TERMS AND DEFINITIONS

No terms and definitions are listed in this document.



5

ABSTRACT



ABSTRACT

To come...



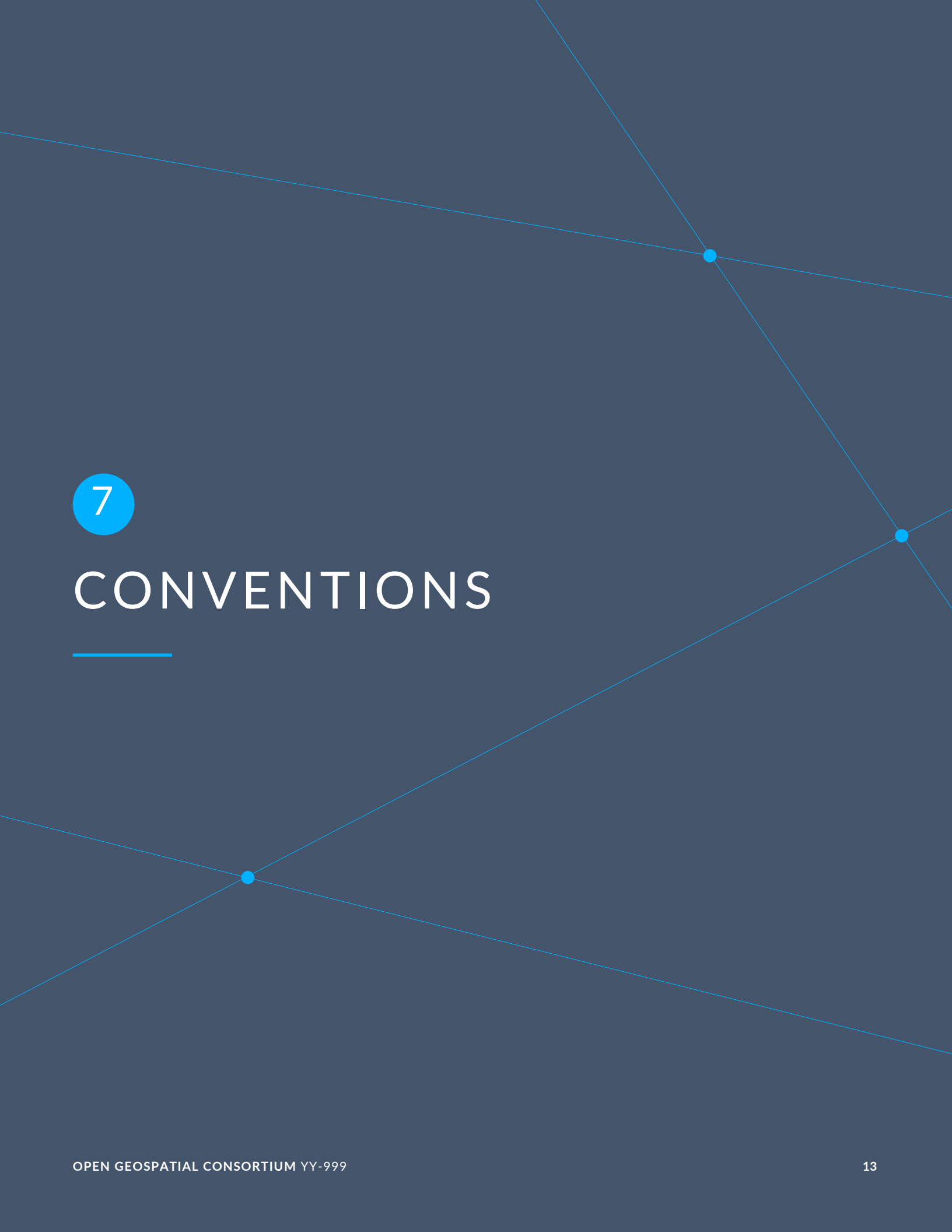
6

KEYWORDS



KEYWORDS

To come...



7

CONVENTIONS



CONVENTIONS



8

INTRODUCTION

OGC [GeoSPARQL](#) is an open standard that enables storage and exchange of spatial data on the Web, based on the Resource Description Framework (RDF). In 2020 the OGC white paper [Benefits of Representing Spatial Data Using Semantic and Graph Technologies](#) was published. It explained the benefits of GeoSPARQL and described the ways in which GeoSPARQL version 1.0 could be improved. In the meantime, [GeoSPARQL version 1.1](#) has been published, having many improvements described in aforementioned white paper, see also [this introduction to GeoSPARQL 1.1](#) [26].

Although not specifically bound, GeoSPARQL 1.1 is mainly geared towards spatial objects having zero to two dimensions. In other words, things that can be displayed on a flat surface. However, for three-dimensional spatial objects both supply and demand are increasing.

Two new versions of GeoSPARQL are in development: 1.2 and 1.3. GeoSPARQL 1.2 will be an adaption of GeoSPARQL 1.1 which enables GeoSPARQL to be published as an ISO standard. Version 1.2 will have no substantive changes with respect to version 1.1. GeoSPARQL 1.3 is expected to have extended capabilities supporting three-dimensional (3D) space. Not only would that make GeoSPARQL more useful for 3D geospatial data, it would also help in industries and knowledge domains that are not mainly focused on geospatial data, like Building Information Modelling (BIM) and Computer Graphics (CG). This paper describes which extensions to GeoSPARQL for 3D are possible, which are most desirable, and how they could be achieved.

This paper consists of three main parts. The first section describes current capabilities of GeoSPARQL: in its current state, GeoSPARQL does allow modelling three-dimensional objects, but with relevant limitations. The second section describes requirements for extended 3D capabilities in GeoSPARQL. This is based on a market consultation, which resulted in a collection of use cases for additional 3D capabilities. By analysing those use cases, as well as current developments within and outside the OGC, insight in the most needed extensions to GeoSPARQL can be obtained. Requirements are then weighed against feasibility; some extensions are easier to achieve than others. The third and final section describes the expected benefits for additional 3D capabilities in GeoSPARQL for different domains and different roles.

This paper should be interesting for the following audiences:

- Current and prospective users of GeoSPARQL;
- Current and prospective implementers of GeoSPARQL;
- Members of related OGC working groups.

A deep understanding of the Semantic Web and graph technologies is not required for readers of this white paper, but some interest in those topics is assumed, as well as familiarity with the aforementioned white paper [Benefits of Representing Spatial Data Using Semantic and Graph Technologies](#). This paper focuses on specifically on 3D capabilities of GeoSPARQL, not its general benefits.



9

MOTIVATION

9.1. Why do we want 3D semantic modeling?

The white paper: “OGC Benefits of Representing Spatial Data Using Semantic and Graph Technologies” describes the beneficiaries and benefits of representing data, including geospatial data, using semantic and graph technologies Abhayaratna2020wp. This section describes the motivation of adding 3D functionality to the GeoSPARQL standard.

While humans can often resolve 3D spatial questions such as “Is cube A inside cylinder B?” — even when the underlying data comes from different systems like GIS and BIM — this typically relies on implicit understanding, conventions, or direct communication between the data creators. For datasets to be meaningfully aligned, the people who created them often need to clarify assumptions, data structures, coordinate systems, or modeling intentions. Without such human-to-human explanation, machines struggle to interpret spatial relationships across heterogeneous sources. A conversion process is required, in which data is made homogeneous for analysis through extraction, transformation, and loading. Based on a homogeneous dataset, conclusions can then be drawn. Performing conversions may result in geometric simplifications, which can limit or even eliminate the functionality of spatial queries Biljecki2019. If one desires digital, automated processing or analysis of 3D spatial questions without requiring conversion, spatial information must be structured in an explicitly machine-readable format. When this is done, initiatives based on federated systems with heterogeneous datasets can obtain computer-generated answers to spatial 3D queries. This requires semantic alignment and standardized geometry processing in machine-driven analysis. In other words, a common vocabulary is needed to semantically define the 3D objects and its 3D spatial function.

Imagine a constructor who wants to use real-time sensor data at a construction site in combination with modelled 3D data and 3D base registration data. By combining this the person wants to visualize the site, perform accurate spatial checks and calculations, and automatically update machine instructions for safe, efficient and autonomous execution”

The system uses 3D GeoSPARQL to integrate the data of the 3D design model, the 3D base registration model and 3D real-time sensor data to:

- Spatially associate sensor data with objects in the design and registration model
- Calculate the permissible tolerances
- Detect the deviations
- Dynamic adjust the design model
- Instruct the machine with the changes
- Visualize the new situation

For both humans and machines, it must be unambiguous what is meant by the queries that are needed to do the job and the resulting answers. It should not matter whether the data is provided in a 3D-GIS, 3D-BIM or 3D-CG standard vocabulary.

9.1.1. A geo-spatial 3D specification would support data consumers by:

- access and understanding of 3D-geometry and 3D-topology from multiple sources;
- the integration of 3D-data geospatial and non-geospatial;
- data quality evaluation of the given 3D-data;
- Geometric and topological description in natural language.

9.1.2. A geo-spatial 3D specification would support data providers by:

- Maximizing the use of 3D-geometry and 3D-topology data;
- Defining multiple 3D topology or geometries;
- Assuring the quality of 3D geospatial data.



10

RELATED WORK

This is still just a dump of resources. To be transformed into actual text.

10.1. CityGML ---

CityGML [koble2005citygml] is an open standardized data model for the exchange of 3D models of cities and landscapes. Since its standardization by OGC, CityGML has seen a wide variety of proposed extensions and alignments to other data standards.

10.1.1. CityJSON and CityRDF

[27] proposed the extension of the CityGML and its accompanying serialization CityJSON as an ontology model. Currently, the CityRDF standardization group works towards the standardization of this CityGML data model in RDF.

The proposed CityRDF model, even though focused on CityGML and 3D building contents, contains valuable data descriptions which could be generalized in a GeoSPARQL 3D approach. One such element is the description of the appearance of a 3D-modeled building using colors or textures, so-called surface materials. In addition, CityGML includes relations of 3D primitives between each other, which could serve as an inspiration for similarly-named functions or properties in the GeoSPARQL 3D ontology model.

[27] describes the CityGML Ontology, which is an ontology model to formalize the CityGML standard towards a representation in OWL. It shows a workflow of identification, classification and selection of the relevant data points needed to describe 3D city models in OWL as derived from building and city models. In addition, it introduces the concept of Level of Information Need, defining the granularity of the information on the various levels of the building and the types of information retrieval — direct/indirect, i.e. information which is commonly obtained directly from a dataset or using a calculation or reasoning approach. The paper exemplifies the concept in a study case for property unit cost and indoor daylight. The contribution outlined here became the basis of the CityGML ontology which can serve as one broad application area for GeoSPARQL 3D.

Following on this publication [22] presents a system architecture and a set of interfaces that can represent city modeling approaches using dynamic geospatial knowledge graphs. The paper proves the viability of a transition between a SQL based system for 3D building management to a SPARQL based system using a SQL2SPARQL Transformer component. It therefore proves the feasibility of employing linked open data technologies for 3D building data and in particular the CityGML model in practice.

[30] introduces CityGML 3.0 with a newly added Core module including a new space concept and Level Of Detail concept. The given publication sets the standard especially for Level of Detail in 3D buildings, which may be considered for adoption as GeoSPARQL 3D concepts too.

In addition, CityGML 3.0 proposes the addition of 3D point clouds for the representation of 3D geometries of physical spaces and space boundaries. A mapping from IFC to CityGML 3.0 is also proposed by these authors.

CityGML 3.0 therefore represents many of the elements that a GeoSPARQL 3D ontology would need to implement on a more abstract level and can give ideas about future query functions to be implemented in the GeoSPARQL query language.

In addition, a master thesis [hansson2024citygml](#) proved the feasibility of not only converting CityGML to a knowledge graph representation, but also its application in triple stores such as Ontop or 3DCityDB. The mapping of CityGML to the knowledge graph was tested using a set of SPARQL queries retrieving different components of building parts. While the thesis could construct a knowledge graph representation from CityGML, it points out that the incorporation of spatial operations is left to future work and could provide a possible field for GeoSPARQL 3D adoption.

Finally, [21] proposed OntoCityGML, an extension to the CityGML ontology <https://www.sciencedirect.com/science/article/pii/S2666546821000574?via%3Dihub> Semantic 3D City Database – An enabler for a dynamic geospatial knowledge graph

Other details and history of Onto CityGML.

CityRDF working group <https://github.com/ogcincubator/cityrdf>

10.2. Medicine & Chemistry

The medicine and chemistry domains contain a variety of usecases for 3D applications.

In particular, 3D metadata of microscopy is a common application field, which yielded for instance the [3D Microscopy Metadata Standards \(3D-MMS\)](#) [32] developed by the BRAIN 3D Microscopy Working Group. This standard helps ensuring that a 3D microscopy dataset is sufficiently described to support its re-use by other scientists not involved in data creation. Its adoption will therefore enable investigators willing to share their data to evaluate and decide which data can be combined.

<https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.13083> Ontology-Based Representation and Modelling of Synthetic 3D Content: A State-of-the-Art Review

[24]

A range of approaches have been proposed to permit semantic representation and modelling of synthetic 3D content. These approaches differ in the methodologies and technologies used as well as their scope and application domains. This paper provides a review of the current state of the art in representation and modelling of 3D content based on semantic web ontologies, together with a classification, characterization and discussion of the particular approaches.

<https://www.inderscienceonline.com/doi/abs/10.1504/IJMSO.2017.087702> A novel ontology for 3D semantics: ontology-based 3D model indexing and content-based video retrieval applied to the medical domain

[25]

This paper presents the most comprehensive formally grounded 3D ontology to date that maps the entire XSD-based vocabulary of the industry standard X3D (ISO/IEC 19775-19777) to OWL 2, complemented by fundamental concepts and roles of the 3D modelling industry not covered by X3D.

10.3. 3D data formats and ontologies

A new GeoSPARQL standard should be compatible with existing approaches to encode 3D data. Semantic technologies should enable users to encode important metadata in the knowledge graph, while retaining the possibility to use already well-known 3D geometry formats either as String literals or file references. [bonduel2019](#) including shows how common widespread geometry formats could be included into knowledge graphs, which could be one starting point to investigate the integration within GeoSPARQL.

10.3.1. STereoLithography (STL)

[STL files](#) describe only the surface geometry of a three-dimensional object without any representation of color, texture or other common CAD model attributes. The STL format specifies both ASCII and binary representations and was invented by the Albert Consulting Group for 3D Systems in 1987. Nowadays it is mainly used for 3D printing purposes, which makes it a plausible candidate for support in GeoSPARQL 3D settings of movable objects such as in cultural heritage. Since the STL format does not contain a geospatial reference, the reference needs to be provided externally if STL files were to be used in a geospatial context. However, STL objects could be used in local CRS contexts as well.

10.3.2. Wavefront OBJ Format (OBJ)

The [OBJ format](#) is a file format developed by Wavefront Technologies for the Advanced 3D Visualizer animation package. Its roots therefore lie in the world of 3D animation. The format represents vertex coordinates, texture coordinates, as well as vertex normals, parameter space vertices and faces of a 3D model. These are all elements which would need to be considered and supported in an upcoming GeoSPARQL 3D release.

10.3.3. Polygon File Format (PLY)

The [Polygon File Format](#) is a format which was designed to store 3D data from 3D scanning environments, which makes it especially widespread in the Cultural Heritage application area. The format consists of a vertex list and a faces list with optional parameters per vertex. Like the

OBJ format, supporting PLY would imply including support for vertices and faces of a 3D model and their relationships in a knowledge graph.

10.3.4. CARARE Metadata Schema

The CARARE metadata schema [d2013carare](#) is a harvesting schema intended for delivering metadata about an organisation's online collections, heritage assets and their digital resources. The strength of the schema lies with its ability to support the full range of descriptive information about monuments, building, landscape areas and their representations. The CARARE metadata schema builds on existing standards and best practice from a number of different countries in Europe and the rest of the world.

10.3.5. IIIF 3D

The International Image Interoperability Framework (IIIF) standard [snydman2015international](#) is currently developing a [specification](#) for the standardization of 3D viewers. The viewer specification includes the visualization of 3D models including lighting positions, a scene graph and annotations in the form of additional 3D objects added to a 3D scene. A GeoSPARQL 3D standard might need to be compatible with, or include a compatibility statement of, converting 3D objects encoded in GeoSPARQL to the IIIF viewer specification. A prototype implementation might be necessary to comply with this specification.

10.3.6. X3D and X3D Ontology

The X3D format [feichtenhofer2020x3d](#), [brutzman2010x3d](#), X3DOM [behr2009x3dom](#) and its Ontology version [brutzman2020x3d](#) provide a data format and a blueprint for how 3D data is handled in computer vision approaches. X3D is an interesting format to investigate for 3D components which need to be modeled within a GeoSPARQL 3D specification. It is a likely candidate for a literal format and might be integrated using interlinking approaches to the already existing X3D vocabulary. However, since X3D was invented with computer vision approaches in mind it does not define any geospatial aspects that could be considered by a GeoSPARQL 3D implementation.

10.3.7. Geometry Metadata Ontology (GOM)

The [Geometry Metadata Ontology \(GOM\)](#) introduces classes for surfaces which are common in the 3D world, but uncommon for 3D geometries, for instance Meshes or NURBS surfaces. The ontology can serve as an inspiration for extensions in GeoSPARQL 3D and may or may not be fully or partly adopted in the process of standardization.

10.3.8. Ontology for Managing Geometry (OMG)

The [Ontology for Managing Geometry \(OMG\)](#) describes states and derivation of geometries which may be useful to the description of data in GeoSPARQL 3D. For example. an extrusion in

3D from a 2D geometry template would have a derivation relation from the 2D geometry which one might want to see modeled in GeoSPARQL 3D.

10.3.9. File Ontology for Geometry Formats (FOG)

The File Ontology for Geometry Formats (FOG) provides a bridge between ontology descriptions of geometries and data file descriptions of 3D contents. Since many 3D datasets are of considerable size, their inclusion into the knowledge graph as String literals might hinder the query performance of said knowledge graph. It therefore stands to reason whether to adopt the same or a similar method of geometry access that FOG provides for GeoSPARQL 3D.

10.4. To consider

<https://link.springer.com/article/10.1007/s10845-023-02246-6> Ontology of 3D virtual modeling in digital twin: a review, analysis and thinking

[33]

To help novice engineers understand and scheme 3D virtual modeling in digital twin for future research and applications, this paper reviews 106 digital twin 3D modeling cases with their characteristics, including deployment targets, purposes & roles, collaborative models, data flows, the autonomy of 3D modeling, fidelity, twinning rates, enabling technologies, and enabling tools.

Open standard for particle-mesh data (openPMD)

The openPMD standard, short for open standard for particle-mesh data files is not a file format per se. It provides guidance for meta-data and naming schemes. openPMD provides naming and attribute conventions that allow the exchange of particle and mesh based data from scientific simulations and experiments. Its primary goal is to define a minimal set/kernel of meta information that allows to share and exchange data to achieve portability between various applications and different algorithms, a unified open-access description for scientific data (publishing and archiving), and a unified description for post-processing, visualization and analysis. If outputs from programs, devices (such as cameras), simulations or post-processed data-sets, contain a minimal set of meta information as provided by openPMD, data can be exchanged between those with minimal effort, with the same tools used for visualization.

[19]

10.5. Cultural Heritage

In the research domain of cultural heritage, 3D models of either cultural heritage artifacts (possibly georeferenced), 3D models of archaeological sites or simply 3D models of ancient buildings are becoming increasingly common.

10.5.1. Use Cases

Use Cases in the Cultural Heritage domain include, but are not limited to, the following main interests:

Visualization: The visualization of 3D models for the presentation of such 3D models, for example in the context of a museum. 3D models may be styled with a particular set of textures or modeled with a specific set of colours to highlight certain relevant aspects. The visualization of 3D models is currently standardized by the IIIF 3D working group [29], which aims to create viewing parameter descriptions that 3D viewers may implement, similar to the specifications of IIIF 2D for images.

Object Annotation: 3D models are seen as the subject of a research question in absence of the original artifact for political, practical or other restrictive reasons. Out of all known methods of the representation of cultural heritage artifacts, 3D models provide the most detail when being delivered as a digital artifact and are therefore very often preferred in a research context. Researchers mark noteworthy aspects of the cultural artifact as 3D annotations [28] which may include surface descriptions, volumes of the 3D model or 3D models which are created and placed adjacent to the to-be-annotated 3D model [20].

Relation of Objects: Objects of a specific collections always exist in a spatio-temporal context. It is important to relate these representations via meaningful relations, so that relevant objects of a collection can be retrieved more easily

AI Applications in Cultural Heritage: Usage for annotated areas on 3D models or their derivations for machine learning classifications Stotzner_2023_ICCV 10.2312/gch.20231157

Knowledge Graphs as Metadata descriptions: With the advent of more 3D models being published, the relevance of their creation parameters [31], their contents and their object metadata increases for the usecases of filtering them and also for the possibly automated selection of suitable cultural heritage metadata for e.g. machine learning classifications. Currently, many metadata standards fulfil parts of the description chain and a unified vocabulary to described data types seems to be missing.

10.5.2. Research applications making use of 3D models in Cultural Heritage

This section discusses research projects with 3D contents based on the technologies they use as elaborated in the previous section.

10.5.2.1. 3D models of cuneiform tablets

Cuneiform tablets from ancient Iran provide an interesting research area, since they combine a 3D artifact with textual imprints that are of interest to a variety of research communities including Assyriologists, Digital Humanists, Computational Linguists and last but not least Computer Scientists. The creation of 3D models of cuneiform tablets provides the best accessibility to the specificities of the original artifact in its absence and 3D scans have been

used by computer scientists as the basis for certain machine learning application tasks, even though to this day only as a provision for 2D renderings of their surfaces. Interests of the research community include the description of interesting features such as cuneiform signs on cuneiform tablet surfaces and their connection to other text contents or other cuneiform artifacts. Knowledge graphs provide machine learning approaches Stotzner_2023_ICCV stotzner2023r with unique opportunities to connect different kinds of data using a unified data format.

To describe 3D meshes, several vocabularies have been developed in the context of the cuneiform studies project:

- MeshSPARQL: A vocabulary to describe essential mesh elements
- Gigamesh Metadata Vocabulary: A vocabulary which describes metadata of a 3D model. The metadata can be generated using the Gigamesh Software Framework
- 3DCAP Vocabularies: An ontology model to describe the creation of a 3D model. It has been applied to different scanning softwares

10.6. IFC and BIM

10.6.1. Industry Foundation Classes (IFC) and BIM

BIM is a paradigm in which object-model definitions — with machine-interpretable semantics — are exchanged, rather than relying on CAD drawings that convey only graphical semantics. The predominant open exchange standard is Industry Foundation Classes (IFC). **IFC is unique in the sense that it allows expressing both geometrical as well as non-geometrical semantics.**

Non-geometrical: taxonomy of objects (building elements, relational constructs, scheduling information) and related properties

Geometrical: approximately 250 constructs related to geometry; even standards with only a geometric focus such as geosparql or collada do not reach that breadth in definitions

IFC has its idiosyncrasies and challenges, but is **unrivaled in terms of market adoption in a highly fragmented industry.**

10.6.1.1. Product model

In IFC, a construction work is decomposed into a set of products. These products can have **multiple representations**. For example, a wall can be described as a solid body as well as a two-dimensional axis. These representations facilitate different views on the same data: an editable line segment or an easily visualized volume. The Object-relational nature of the IFC EXPRESS schema allows intricate relationships such as a representation context that communicates

additional intent for the representation or presentation styles that can be granularly assigned to individual faces.

At the same time, such a product separates the **placement** (an hierarchical transformation) from the actual geometry definition. The consequence of this is that in spite of its object-relational nature, IFC product representations cannot be used for building-level topological relationships between solids, because even if two solids are touching in 3D, the fact the the placement is externalized out of the geometry definition (or the fact that faces are constructed procedurally and do not exist explicitly), means that the two faces cannot be opposite oriented twins. As such, relational geometric constructs such as space boundaries are provided as additional supplementary geometries. (Identifying faces in procedural geometries is also challenging (topological naming problem), so using geometric semantic overlays to annotate potentially procedural geometries makes sense)

In principle, the IFC schema has been designed in a modular fashion with independent modules for, for example, geometry, materials and meta-data. However in other cases, **semantics and geometry are intertwined** such as tapered extrusions (lofts) where the begin and end profile of a duct carry important semantics.

IFC also allows for **decomposition**, where a whole is aggregated into multiple parts for richer semantics. This allows for example to connect materials and meta-data to the frame and the glazing separately, while still being able to identify the aggregate as a single window. This is not used as frequently, partially due to inability to efficiently instantiate such aggregates as geometry instances.

10.6.1.2. Evolving views on geometry

IFC is heavily influenced by the ISO 10303 (STEP) family of standards, but over time adopted its own geometric paradigms:

- Procedural geometry and boolean operations became less prominent with the adoption of ReferenceView in IFC4. Tessellated geometry definitions were added for more compact exchange.
- **Infrastructure definitions** were added with precise mathematical transition curves and a composition of a horizontal, vertical and cant (inclination) profile.
- IFC5 with an **explicit** (most likely triangulated) geometry schema at the core, with semantic overlays to encode the same procedural semantics as a non-mandatory or use-case specific layer. Heavily inspired by USD with layer-based composition for collaborative exchange.

Especially the handling of **tolerances** means that the standard cannot effectively prescribe a consistent outcome in all cases. Tolerances are needed for BRep model with non-linear underlying geometry and/or fixed precision coordinate values, e.g higher degree nurbs curves are typically intersected with numerical approximation, so a vertex that connects two of such curves needs to have seen as a sphere with the local tolerance as its radius. This tolerance is also applied to boolean operations: an subtraction volume can be slightly inwards of the first operand but is still expected to pierce through the volume and increase surface genus. This contrasts

with the desire of using IFC as a legal basis in contracts. NB Tolerances stand in the way of using existing approaches for SFA geometry predicates such as PostGIS+SFCGAL which is based on arbitrary precision boolean logic as implemented in CGAL without tolerances.

10.6.1.3. Use cases

The most successful use case on BIM data is **coordination and visualization** where multiple aspect models are geometrically overlaid in order to find issues, which are then communicated to the authoring software where they are addressed. This approach works, because it respects that individual disciplines all have their own specialistic software.

Design to design workflows are much harder to realize, although some Model View Definitions have been developed on top of IFC that enable the transfer of design intent in specific and constrained scenarios, such as precast concrete and structural steel.

Long-term preservation of building information is difficult because of the fact that IFC models are difficult to mutate, because they are so explicit and don't contain the vendor-specific design intelligence. Therefore native software cannot always re-import IFC models, but also the native models degrade over time because of the need to migrate to newer editions of the software. Software that can directly operate on IFC to make modifications is still experimental.

BIM-GIS integration is challenging because it requires familiarity with both domains on where to draw the line between euclidean and non-euclidean geometries and acceptable error metrics.

Simulations on IFC building models are often challenging because the 'bag of individual elements' does not provide a good foundation higher order topological representations required for flow-of-energy type of simulations. For e.g thermal simulation a topological view of space boundaries is required. They have been added as secondary set of ternary relationships, but usage of more specific-purpose and simpler schemas sees still more usage in industry. In general, IFC models are created for a specific purpose and wide-spread usage of those models in neighbouring domain remain challenging because modelling for those neighbouring purposes requires alignment on the worldviews and levels of detail that is often beyond the scope in which such models are procured.

10.6.1.4. Implications and questions:

- Euclidean / non-euclidean; is a CRS required?
- Separate representation+placement → enables efficient reinstantiation, but hinders topological relationships because you require the pair of placement+geometry to locate in space
- Geometry as leaf-values or object-relational model : cannot encapsulate geometry into a single literal, but allows for richer semantics
- BRep model (topology + geometry + orientation + location) vs polyhedral model (e.g halfedge) vs explicit loops of point coordinates
- Procedural vs implicit (e.g constraints) vs explicit (polyhedra)

- Tolerances
- Decomposition inside or outside of the 'geometry ontology'
- Are infra geometries (hor + ver alignment + cant, for positioning and sweeps) in scope?

10.7. Implementations

10.7.1. CGAL → SFCGAL → PostGIS

10.7.2. OpenCASCADE

10.7.2.1. OpenCASCADE-inspired BRep ontology

Perzylo, A., Somani, N., Rickert, M., & Knoll, A. (2015, September). An ontology for CAD data and geometric constraints as a link between product models and semantic robot task descriptions. In 2015 IEEE/RSJ international conference on intelligent robots and systems (IROS) (pp. 4197-4203). IEEE.

<https://ieeexplore.ieee.org/abstract/document/7353971>

[23]

10.7.2.2. Topologic

Jabi, W., & Chatzivasileiadi, A. (2021, January). Topologic: exploring spatial reasoning through geometry, topology, and semantics. In Formal Methods in Architecture: Proceedings of the 5th International Symposium on Formal Methods in Architecture (5FMA), Lisbon 2020 (pp. 277-285). Cham: Springer International Publishing.

<https://topologic.app/>

[2]

10.7.3. BRep vs mesh/polyhedron

BRep

- Curved surfaces
- Topology: connected components as shells, solids with inner voids, etc.

- Clean APIs due to inheritance: e.g `fn extrude(Topo) → Topo`, for `vertex → edge`; `edge → face`; `face → solid`; `solid → solid`
- Extra indirections: `edge → vertex[] → point`
- Depending on implementation can be inefficient, e.g outer wire of face not explicitly marked need to be checked wrt infinite point
- Data integrity and validation a bit harder

Mesh/polyhedron

- Potentially fewer indirections
- Triangle meshes robust and well understood
- Many different data models though, e.g half-edge (only manifold), indexed faceset (no adjacency info), winged/quad/radial edge



11

CURRENT CAPABILITIES

11.1. GeoSPARQL

GeoSPARQL is the most common geospatial extension of SPARQL. It was accepted as an OGC standard in 2012 and revised as GeoSPARQL 1.1 in 2024.

According to the standard document, “The OGC GeoSPARQL standard supports representing and querying geospatial data on the Semantic Web. GeoSPARQL defines a vocabulary for representing geospatial data in RDF, and it defines an extension to the SPARQL query language for processing geospatial data”.

11.1.1. Requirements addressed

In order to define which capabilities GeoSPARQL needs to adopt for full 3D compatibility, we first take a look at GeoSPARQL 1.1 current capabilities with regards to 3D.

11.1.1.1. Dimensionality

In this white paper, when we discuss three dimensional data, we mean three dimensional geometries; instances of the class `geo:Geometry`. GeoSPARQL 1.1 defines three different properties for geometry that have to do with dimensionality:

1. `geo:dimension`, or *topological dimension*: This is the number of perpendicular directions in which a geometry extends. A point, for example, extends in no direction, so its topological dimension is 0. A line has a length, but no width or height. Its topological dimension is 1. A cube has a topological dimension of 3.
2. `geo:coordinateDimension`: In the geometric model that GeoSPARQL uses, points are the basic building blocks of geometry. A point geometry can have a different number of coordinates, depending on the space in which it is placed. A point on a flat map, or on the surface of a sphere, needs two coordinates. The *coordinate dimension* is the number of coordinates in the points that define a Geometry.
3. `geo:spatialDimension`: In order to support linear referencing, geometries in GeoSPARQL can have a measure value. This is a relative position along a line or a curve, and does not extend the Geometry in space. The *spatial dimension*, therefore, can be used to denote the number of coordinates in a point, excluding the measurement value.

The extended capabilities for GeoSPARQL discussed in this white paper concern the cases where the *spatial dimension* (which can be expressed with property `geo:spatialDimension`) of geometry has the value 3.

The table below shows the values of the three dimension properties for some Geometries (expressed as Well Known Text (WKT)).

Table 1

GEOMETRY (WKT)	GEO:DIMENSION	GEO:COORDINATEDIMENSION	GEO:SPATIALDIMENSION
point (4 15)	0	2	2
point Z (4 15 3)	0	3	3
linestring (4 15, 13 2)	1	2	2
point M (4 15 60)	0	3	2
point ZM (4 15 3 60)	0	4	3

11.1.1.2. Vocabulary

GeoSPARQL version 1.1 consists of a Core module, a Topology Vocabulary extension, a Geometry extension, a Geometry Topology extension, an RDFS Entailment extension, and a Query Rewrite extension. Each of these modules will be briefly explained below.

11.1.1.3. Core Module

GeoSPARQL version 1.1 includes a CORE module. This defines the basic classes, relationships, and literals. These classes are suitable for both 2D and 3D.

11.1.1.3.1. Topology Vocabulary Extension

This extension provides standard definitions for spatial relationships that can exist between one spatial object and another.

Relations between geometries have been defined using three different sets of rules:

- Simple Features Relations
- Egenhofer Relations
- Region Connection Calculus RCC8

These topological definitions specify topological relations in a 2D-context.

11.1.1.3.2. Geometry Extension

GeoSPARQL 1.1 geometry module defines a class `Geometry` as a subclass of `SpatialObject`.

An instance of `Geometry` is not restricted to two dimensions.

A fine-grained classification of `Geometry` can use the Simple Feature Vocabulary which extends the class `Geometry` with further types, such as *Point*, *Polygon* etc.

The Simple Features vocabulary allows for the definition of 3D variants of:

- (Multi)Points
- (Multi)LineStrings
- (Multi)Polygons
- Polyhedralsurface

It does not include commons 3D primitives, such as *Cube* or *Mesh* surfaces which are integral parts of 3D representations. Nor does it include extruded geometry.

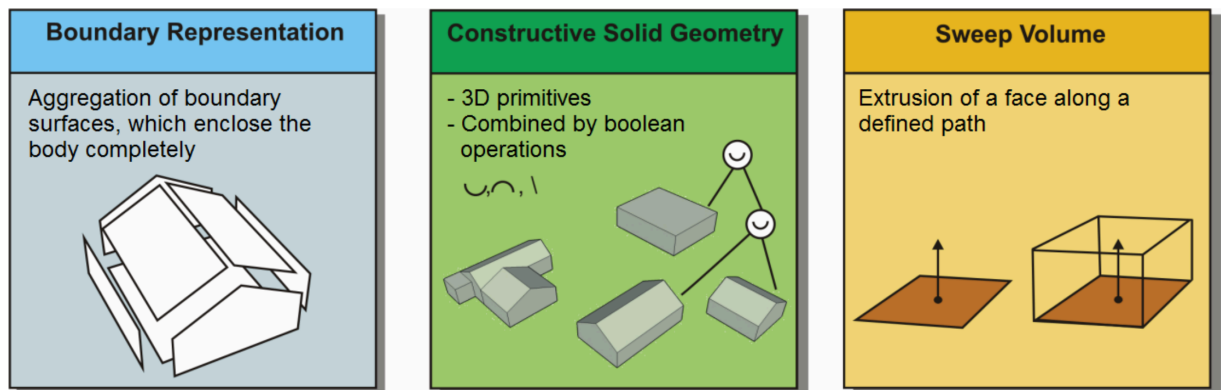


Figure 1 – Possible approaches for representing 3D objects in IFC

Concerning metadata of 3D models, GeoSPARQL 1.1 provides properties which can be reused in 3D contexts. In particular, the properties are:

- `geo:hasVolume`
- `geo:hasMetricVolume`

Further 3D-related metadata properties such as projection matrices are not part of the current GeoSPARQL 1.1 standard.

A first requirement for 3D support is the ability to save 3D data in a knowledge graph.

The Geometry extension enables geometries to be defined within the GeoSPARQL framework. Supported geometry formats include RDF literals in WKT, GML, GeoJSON, and KML.

These literals are investigated for the storage of 3D data.

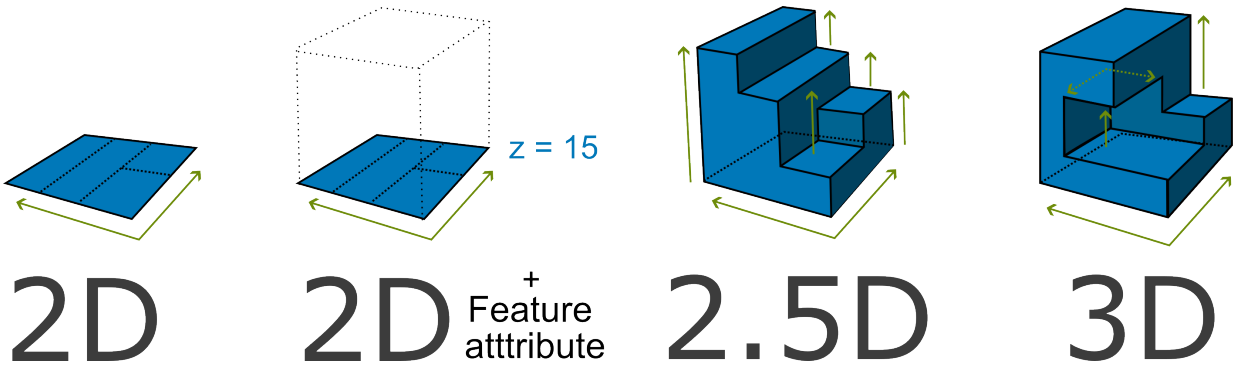


Figure 2 – Difference between 2D, 2D with height attribute 2.5D and 3D geometry

Table 2

Literal Type	Z-Coordinate Supported	Z-Coordinate Supported + height attribute	2.5D	3D
WKT Literal	Yes	Yes	Yes	Yes
GML Literal	Yes	Yes	Yes	Only with extension Schema
KML Literal	Yes	Yes	Yes	As import from COLLADA
GeoJSON Literal	Yes	Yes	Yes	Yes
DGGS Literal				

GeoSPARQL 1.1 also does not restrict the usage of coordinate reference systems with 3D support. There are currently almost 300 coordinate reference systems in the database epsg.io which can be used to describe 3D data encoded in the GeoSPARQL graph literals listed above.

GeoSPARQL already supports linking multiple geometries (e.g., 2D and 2.5D) to a single feature. We aim to extend this with 3D geometries.

For example, a tree in the real world may be represented in databases with both a 2D and a 3D geometry.

This should not require duplication of the tree entity – there can be one tree instance with both geometries linked, avoiding redundancy in the data model.

11.1.1.4. Query functions with 3D support

GeoSPARQL 1.1 functions currently do not offer fully-featured 3D support. However, there are functions which may take into account the Z coordinate, if they are available.

Table 3

GeoSPARQL function	Z-Coordinate Supported	2.5D	3D
geof:is3D	Yes	Yes	Yes
geof:minZ	Yes	Yes	Yes
geof:maxZ	Yes	Yes	Yes

These functions check for the presence of Z coordinates or filter out maximum and minimum Z coordinates of the given geometry.

11.1.2. Adoption of GeoSPARQL 1.1

11.1.2.1. GeoSPARQL Alignments

See Annex A and Annex B (Text to write)

11.1.2.2. Geometry Topology Extension

Another extension is the Geometry Topology extension. This provides query functions that return relationships between different geometries based on the topology vocabulary extension.

11.1.2.3. RDFS Entailment Extension

The RDFS Entailment extension provides rules for reasoning over geometries. Based on specific statements, additional information can be inferred. This kind of logical inference can also be applied to geometry. GeoSPARQL includes logic for reasoning over simple features geometries.

11.1.2.4. Query Rewrite Extension

GeoSPARQL allows queries such as whether “Feature A” is located within “Feature B” using its vocabulary. The Query Rewrite extension specifies a RIF rule that enables query rewriting. However, this extension does not support the rewriting of 3D queries.



12

REQUIREMENTS FOR GEOSPARQL 3D

12.1. 5.1 Existing implementation of 3D geometry in GeoSPARQL

This section provides an overview of feedback received on the current version of the GeoSPARQL standard (version 1.1) regarding 3D usage. It helps to identify some of the barriers to use, and to outline requirements that have not been addressed that may encourage greater uptake.

12.1.1. Proposed extensions for GeoSPARQL 3D

12.1.1.1. Extension 1: 3D representations

12.1.1.1.1. GitHub Issue URI

<https://github.com/opengeospatial/ogc-geosparql/issues/583>

12.1.1.1.2. Category

Semantic improvement

12.1.1.1.3. Description

GeoSPARQL should include ways to represent 3D data in a knowledge graph.

3D data should be included in common 3D formats and 3D data should be includable as a text literal and a file representation.

Some common formats which could be considered for inclusion are:

- [Polygon File Format \(PLY\)](#)
- [Wavefront OBJ Format \(OBJ\)](#)
- [GLTF Format \(GLTF\)](#)
- [X3D Format](#)

12.1.1.2. Extension 2: Topological relations of 3D geometries

12.1.1.2.1. GitHub Issue URI

<https://github.com/opengeospatial/ogc-geosparql/issues/416><https://github.com/opengeospatial/ogc-geosparql/issues/581>

12.1.1.2.2. Category

Semantic improvement

12.1.1.2.3. Description

GeoSPARQL should include ways to represent relations between 3D geometries and relations between 3D geometries and geometries of lower dimensions. The relations should be expressible in property relations and should be queryable using SPARQL extension functions.

- Topological relations in 3D

The topological possibilities in a 3D space differ from those in 2D space. The mathematical formulations used to verify topological relationships in 2D are different from those required in 3D. Furthermore, the algorithms used to analyze topological relationships also differ, as the sequence of analyses needed to identify and evaluate these relationships is not the same in two- and three-dimensional space. As a result, topological analysis in 3D requires a fundamentally different approach from that used in 2D.

The example below illustrates the differences in the possible topological relationships in two- and three-dimensional space for surfaces and lines. All relationships drawn in (2D) exist in both 2D and 3D environments. The relationships illustrated in 3D can only exist in a true 3D environment. It should be noted that not all theoretically possible topological relationships have been assigned a descriptive name. Some relationships are identified only by a numerical identifier within the underlying topological model and therefore cannot be queried directly using named predicates (such as Disjoint, Intersects, or Contains). Investigating which of these unnamed relationships are relevant and should be exposed as explicit query operators in a 3D environment could be an interesting direction for future research.

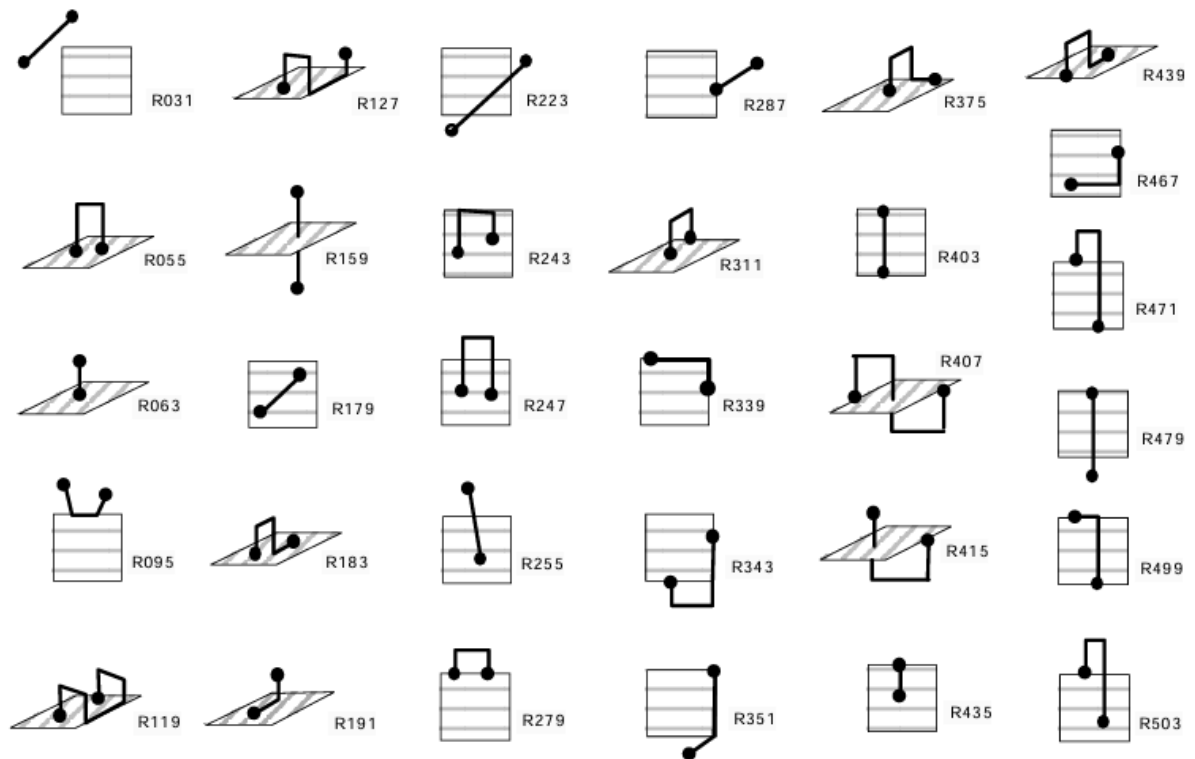


Figure 1: Surface and line in $\mathbb{R}^3$: 31 relationships (19 in $\mathbb{R}^2$, face drawn as reactangle)

Figure 3 – Surface and line topological relations in 2D and 3D space

Disjoint (R031) Inside (R223) Touch (R287, R279) Covers (R439, R435) Crosses (R159, R243, R247, R255) Equal (R403, R407, R415) CoveredBy (R467, R471, R479) Contains (R179, R183, R191) Overlaps (R499, R503) Nameless topological situation (R127, R375, R055, R063, R339, R095, R343, R119, R351)

The semantic definition of a topological relationship, for example disjoint, is identical in both two- and three-dimensional spaces, the outcome of a disjoint function depends on the space in which the geometries are evaluated.

A Query3D evaluates the topological relationship directly in the original three-dimensional Euclidean space. In contrast, a Query2D first projects the geometries onto a two-dimensional plane and subsequently evaluates the topological relationship in this projected space. Consequently, a Query2D does not determine whether two 3D geometries are disjoint in 3D space, but whether their 2D projections are disjoint in 2D space.

As a result, the same pair of geometries may produce different outcomes depending on whether they are evaluated using Query2D or Query3D. Two objects may be disjoint in three-

dimensional space while their projections overlap, causing Query3D to return true for Disjoint and Query2D to return false.

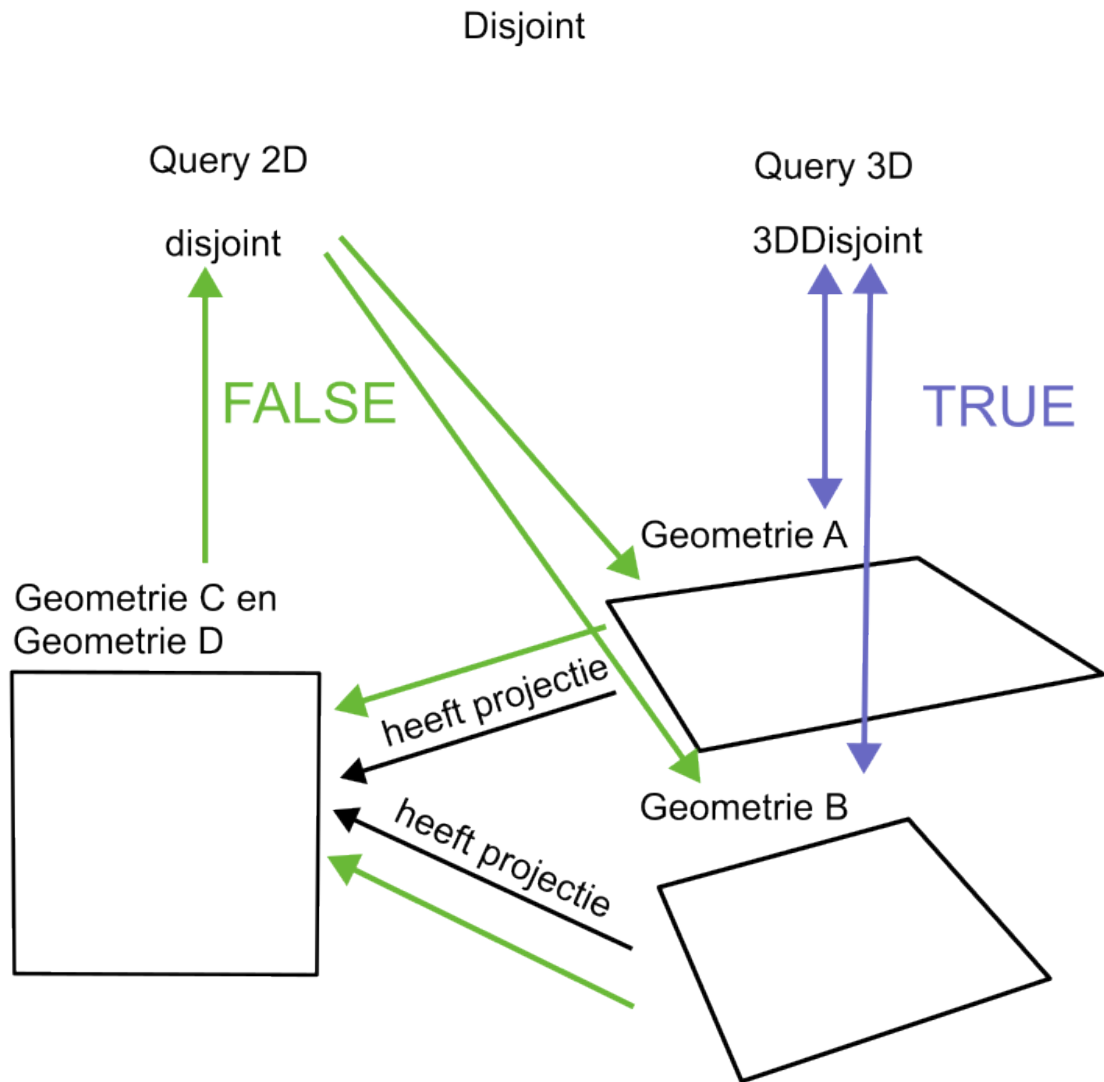


Figure 4 – Disjoint with Query2D and Query3D functions

The example assumes that both geometries are in the same dimension (2D). It is also common to analyze the topological relationship between geometries of different dimensions 0D, 1D, 2D or 3D. Such comparisons require both geometries to be represented in the same spatial dimension before a meaningful topological analysis can be performed.

This can be achieved in one of two ways:

- embedding the 2D geometry into 3D space; or
- projecting the 3D geometry into 2D space.

It is recommended that these transformations are performed explicitly and with a consciously selected Z-value.

For example, when ST_Force3D is applied to a 2D geometry, most implementations automatically assign a Z-coordinate of 0, thereby producing a valid 3D geometry. This default value does not necessarily indicate that the geometry is actually located at an elevation of zero. The value merely serves as a placeholder to satisfy the requirements of a three-dimensional coordinate system. After conversion, it is no longer possible to distinguish whether a Z-value of 0 originated from the source data or was automatically introduced during the conversion process. For applications in which elevation is meaningful, this ambiguity may lead to incorrect interpretations or topological analyses. Explicitly specifying the Z-value during the conversion makes this modeling decision transparent and reproducible.

When a Query2D is requested for geometries that exist only as 3D objects without an explicitly defined projection, the system should indicate that the question is not directly applicable. Instead, it should require the user to specify or confirm the projection to be used before performing the topological evaluation. This would make the dimensional context explicit and help prevent misinterpretation of the query results.

12.1.1.3. Extension 3: Directional relations of 3D geometries

Topological relations describe the relationships between objects independently of perspective. Relative directional relations, on the other hand, describe relationships between objects that depend on a reference frame or viewpoint. In 2D, relations such as left of, above, and beside are often sufficient. In 3D, additional relations become relevant, such as above/below and in front of/behind

Relative directional relations can also provide context in 2D, but many spatial situations can already be described using topology (e.g., inside, touching, overlapping). In 3D environments, however, directional relations become more important because height and depth introduce additional dimensions that cannot be distinguished by topology alone.

For example, describing that a “damage location is underneath a bridge deck” requires both a topological relation (e.g., a 3D contains relation) and a relative directional relation (e.g., below).

This combination of relative location and topological relations has been described by Anne Göbels and Jakob Beetz in the Reloc ontology. [Reloc ontology](#).

This ontology brings natural language closer to the 3D model by formalizing descriptions of relative directions and topological relationships.

12.1.1.3.1. GitHub Issue URI

none

12.1.1.3.2. Category

Semantic improvement

12.1.1.3.3. Description

GeoSPARQL should be able to define or function directional relations as above, on the right of, front, etc.

12.1.2. Proposed extensions for GeoSPARQL 3D

12.1.2.1. Extension 4: Routing

12.1.2.1.1. GitHub Issue URI

<https://github.com/opengeospatial/ogc-geosparql/issues/572>

12.1.2.1.2. Category

Semantic improvement

12.1.2.1.3. Description

GeoSPARQL should be able to query for the shortest route through a 3D space made up of smaller, connected 3D spaces when topological relations are not available.

12.1.2.2. Extension 5: Appearance of 3D geometries

12.1.2.2.1. GitHub Issue URI

- <https://github.com/opengeospatial/ogc-geosparql/issues/584>
- <https://github.com/opengeospatial/ogc-geosparql/issues/592>

12.1.2.2.2. Category

Semantic improvement

12.1.2.2.3. Description

GeoSPARQL should include ways to represent materials and textures of 3D geometries, so that geometries can be styled accordingly.

Materials include:

- Colors of surfaces with light diffusion parameters
- Images as textures, which are associated with surfaces of the 3D object

12.1.2.3. Extension 6: Multi-component 3D geometries

12.1.2.3.1. GitHub Issue URI

<https://github.com/opengeospatial/ogc-geosparql/issues/591>

12.1.2.3.2. Category

Semantic improvement

12.1.2.3.3. Description

GeoSPARQL should include ways to define multi-component 3D geometries, whereas each component expresses its own semantics. For example, parts of a building could have different semantics according to the function of the building components and would be classified as such in an RDF graph.

12.1.2.4. Extension 7: Positioning of 3D geometries

12.1.2.4.1. GitHub Issue URI

- <https://github.com/opengeospatial/ogc-geosparql/issues/587>
- <https://github.com/opengeospatial/ogc-geosparql/issues/588>

- <https://github.com/opengeospatial/ogc-geosparql/issues/589>
- <https://github.com/opengeospatial/ogc-geosparql/issues/591>

12.1.2.4.2. Category

Semantic improvement

12.1.2.4.3. Description

GeoSPARQL should include ways to position 3D geometries in a 3D space. Commonly 3D geometries are rotated, translated and scaled using commonly defined operators in computer graphics. Similar operations are needed for the relative positioning of 3D objects in GeoSPARQL, as properties and potentially as functions.

12.1.2.5. Extension 8: Alignments of GeoSPARQL 3D

12.1.2.5.1. GitHub Issue URI

- <https://github.com/opengeospatial/ogc-geosparql/issues/590>
- <https://github.com/opengeospatial/ogc-geosparql/issues/574>
- <https://github.com/opengeospatial/ogc-geosparql/issues/571>

12.1.2.5.2. Category

Semantic improvement

12.1.2.5.3. Description

GeoSPARQL 3D should be aligned to other vocabularies and standard which currently provide 3D support in different knowledge domains. Especially alignments to ifcOWL and the X3D vocabulary would position the GeoSPARQL vocabulary as a link between these different standards.

12.1.2.6. Extension 9: Alignments of Engineering CRS to Geospatial CRS

12.1.2.6.1. GitHub Issue URI

<https://github.com/opengeospatial/ogc-geosparql/issues/586>

12.1.2.6.2. Category

Semantic improvement

12.1.2.6.3. Description

GeoSPARQL 3D should provide the opportunity to align a local coordinate system in which most 3D geometries are defined with a coordinate reference. While this work might only be partially done within the scope of GeoSPARQL itself, GeoSPARQL should be aligned with the emerging Ontology CRS developments of OGC and provide necessary functions or properties to create the link.

12.1.2.7. Extension 10: Geometry Extrusion

12.1.2.7.1. GitHub Issue URI

- <https://github.com/opengeospatial/ogc-geosparql/issues/556>
- <https://github.com/opengeospatial/ogc-geosparql/issues/547>

12.1.2.7.2. Category

Semantic improvement

12.1.2.7.3. Description

GeoSPARQL 3D should provide the opportunity to extrude 2D geometries to 3D geometries and vice versa.

12.1.2.8. Extension 11: Geometry Attributes

12.1.2.8.1. GitHub Issue URI

- <https://github.com/engeospatial/ogc-geosparql/issues/568>
- <https://github.com/engeospatial/ogc-geosparql/issues/550>
- <https://github.com/engeospatial/ogc-geosparql/issues/549>
- <https://github.com/engeospatial/ogc-geosparql/issues/548>
- <https://github.com/engeospatial/ogc-geosparql/issues/558>

12.1.2.8.2. Category

Semantic improvement

12.1.2.8.3. Description

GeoSPARQL 3D should provide functions and properties that describe essential properties of a 3D Geometry such as its minimum and maximum height, width and depth and its CompactnessRatio.

12.1.2.9. Extension 12: Non-topological Query Functions – 3D Extension

12.1.2.9.1. GitHub Issue URI

- <https://github.com/engeospatial/ogc-geosparql/issues/556>

12.1.2.9.2. Category

Semantic improvement

12.1.2.9.3. Description

GeoSPARQL 3D should provide the opportunity to execute non-topological query functions on 2D and 3D geometries commonly used in geospatial databases. Proposed extensions include following functions:

- geometry extrusion to the specified line segment
- geometry extrusion to the specified height
- spatiotemporal geometry extrusion to the specified line segment with specific start and end time

12.2. 5.2 3D Geometry available in IFC

IFC is a standard with a rich set of geometric primitives, with core concepts defined in the geometry, presentation, material, topology and profile resource subschemas. But also outside of these modules, there are various concepts that have geometrical implications such as openings and space boundary relationships in the productextension subschema. This listing below tries to provide a comprehensive overview broken down per the categories below:

- Relational; typically IfcRelationship between elements subtypes, but also clearance geometry which is a potential additional shape representation identifier
- Product-level; resources applicable the IfcProduct such as material-based styling; openings
- Item-level; the individual items that make up representation of the element; which are in turn potentially affected by product-level features such as openings and styling
- Context; provides additional information on individual items such as precision, target scale and semantic use of the geometry
- Geo-referencing; Prevalence is measured as a text-based scan through a set of 1000 files (500 for novel constructs). The number indicates percentage of model featuring this entity; and then the average number of instances for those models that contain any. See second table for more details.

Table 4

LEVEL	CONCEPT	PREVALENCE	NOTES	RELATED TERMS FROM ISO / OGC (SFA, GEOSPARQL, GML, ...)
Relational	Space boundaries (connection geometry)	12.6% (1676)	IfcRelSpaceBoundary: Surfaces representing adjacencies between spaces and building elements primarily intended for thermal simulation.	geof:sfTouches
Relational	Interferences	1.4% (3)	IfcRelInterferesElements: Primarily intended for modelling clash detection outcomes and over- and underpasses in infrastructure models. Likely has seen little use in practice.	ST_Intersects
Relational	Metric constraints	0%	IfcMetric: distances, parametric expressions between properties and geometry; supported in the schema (has limitations on indexing into unordered aggregates) and rarely used in practice.	ST_Distance owl:sameAs rdfs:seeAlso
Relational	Clearance geometry	0%	IfcShapeRepresentation[Representation 'Clearance']: Dedicated shapes encoding minimum clearances (e.g., around equipment, maintenance envelopes, door swings); a separate representation with its own context	
Product	Relative positioning	97.6% (1298)	IfcLocalPlacement provides hierarchical local coordinate systems for products. Informal propositions are in place to define a placement hierarchy that follows the project decomposition structure	gml:AffineCS
Product	Linear positioning	4.8% (527)	IfcLinearPlacement provides linear positioning that derives an orientation and positioning frame from a length measure along an (IfcAlignment) curve	ISO 19148:2021
Product	Grid placement	0%	IfcGridPlacement provides placement by means of the computed intersection between	

LEVEL	CONCEPT	PREVALEN	NOTES	RELATED TERMS FROM ISO / OGC (SFA, GEOSPARQL, GML, ...)
			two grid axes with underlying curve geometry	
Product	Openings	49.1% (168)	IfcRelVoidsElements + IfcOpeningElement/ IfcVoidingFeature models a subtractive boolean operation applied to host elements (walls, slabs, etc.); in principle extensible to other feature kinds such as IfcProjectionElement. In IFC2X3 CoordinationView these were to be processed by the importer. In IFC4 Reference View, in principle, these are precomputed at export time.	ST_Difference
Product	Material association	71% (324)	IfcRelAssociatesMaterial, layered / list / constituent / profile-based Links product geometry to material models: layered elements, material lists, constituents, profile-based materials. Especially layered materials (IfcMaterialLayer Set[Usage]) have geometric implications because it semantically slices the geometry by means of planes defined from offsets along a reference curve (for vertical elements) or along the extrusion direction.	
Product	Parametric geometry as geometry-related semantics via property sets (e.g. Pset_MechanicalFastener, Pset_WindowLiningProfile, Pset_ProvisionForVoid, most quantities Qto_*)	29% (73)	Semantic parameters that can be used to drive or reconstruct geometry when explicit representations are absent.	
Product	Product-level composite curves defining the	4.2% (504)	Decomposition of IfcAlignment into IfcAlignmentSegment with parametric geometry (typically	OGC LandInfra/InfraGML alignment model

LEVEL	CONCEPT	PREVALEN	NOTES	RELATED TERMS FROM ISO / OGC (SFA, GEOSPARQL, GML, ...)
	'business-logic' of parametric alignment segment nesting within a curve		redundant with item-level curves).	
Product	Aggregation of product into assemblies: composition (IfcElementAssembly, nesting (ports, alignment segments)	96.9% (29)	Product-level aggregation (e.g., ports, sub-elements, segments)	
Item	Basic 0D/1D/2D items: points, curves, faces (IfcCartesianPoint, IfcPolyline, IfcCompositeCurve, IfcFace)	99.2% (34194)	Simple representations or underlying points, polylines, composite curves (including circular/arc segments), and faces used in higher-level geometry.	SFA: Point, LineString, MultiLineString, Polygon; GML: gml:Point, gml:Curve, gml:LineString
Item	Profiles: parametric (e.g I shape) / arbitrary / derived / composite (IfcProfileDef, IfcArbitraryProfileDef, IfcDerivedProfileDef, IfcCompositeProfileDef) [Note: not strictly an independent IfcRepresentationItem]	76.9% (763)	Cross-section definitions used by sweeps and structural members; parametric catalog profiles and arbitrary section shapes.	
Item	Annotation geometry: text literals, hatches and fills	11.6% (101)	IfcTextLiteral, IfcFillAreaStyle etc. 2D annotation geometry, symbolization and fills; mostly for documentation, not primary 3D model.	
Item	Explicit (BRep) representations of 3D faceted solids and surfaces	43.9% (876)	IfcFacetedBrep	SFA: PolyhedralSurface
Item	Explicit (BRep) representations of 3D curved solids and surfaces	6.5% (112)	Advanced BRep: with (e.g) NURBS faces (IfcAdvancedFace)	

LEVEL	CONCEPT	PREVALEN	NOTES	RELATED TERMS FROM ISO / OGC (SFA, GEOSPARQL, GML, ...)
Item	Explicit mesh/ polyhedron representations (called tessellation)	12.6% (163)	IfcTessellatedFaceSet IfcTriangulatedFaceSet IfcPolygonalFaceSet Indexed face sets representing triangulated or polygonal surfaces/solids; compact storage via point lists and indices.	SFA: PolyhedralSurface gml: TriangulatedSurface
Item	TIN / surface models	2% (1)	IfcTriangulatedIrregularNet Specialized tessellated representation for terrain and horizontal surfaces (single f(X, Y)→Z for any X,Y); typical for infrastructure and terrain models.	gml:Tin
Item	Swept / procedural solids: extrusion, revolution, sweeps along curve (disk, arbitrary, fixed reference), sectioned solids and surfaces	75.5% (703)	3D solids defined by sweeping profiles along vectors or curves: straight/linear extrusions, revolutions, sweeps along curves (swept disk, arbitrary orientation, fixed reference), tapered forms, sectioned solids/surfaces. Note that this is a fairly large category with many items and combinatorial possibilities lumped into one.	
Item	Axis geometry	47.6% (321)	Bounded curve geometries as (often) a secondary representation describing the longitudinal direction of a solid or element. Material layer sets are defined relative to this axis.	
Item	Boolean operations (CSG trees)	30.6% (122)	Solids defined as binary trees combining operands via union, intersection, difference; often nested and require recursive traversal.	SFA: ST_Union, ST_Intersection, ST_Difference
Item	CSG primitives	1.7% (8)	IfcCsgPrimitive3D Are only part of the CSG expressions, but include things like cylinder and sphere which are 'harder' to produce otherwise (as revolutions)	
Item	Surface-only geometry for	31.8% (1521)	Surfaces used without explicit solid closure, e.g. for space	gml:Surface

LEVEL	CONCEPT	PREVALEN	NOTES	RELATED TERMS FROM ISO / OGC (SFA, GEOSPARQL, GML, ...)
	connection / space boundaries and advanced face surfaces (extrusion, revolution, bounded, NURBS)		boundaries or analytical/ architectural surfaces.	
Item	Styling by item as a whole (IfcStyledItem, IfcSurfaceStyle, IfcCurveStyle)	68.3% (1458)	Assignment of colors, line styles to geometric items or tessellated faces as a whole	
Item	Granular styling by sub-item (IfcSurfaceStyle, IfcIndexedColourMap)	-	E.g the application of a different color to a face within a shell, or per-face colors in tessellations using IfcIndexedColourMap, or retaining distinct surface styles after applying boolean operations	
Item	Styling by item: Textures	0.8% (2)	IfcSurfaceStyleWithTextures and three subtypes of IfcSurfaceTexture for a specialization of the above with texture as a binary blob, a compressed inline literal or an external URI reference. The complexity of supporting textures is mostly related to handling and defining texture coordinates (UV) on the procedural geometries (such as sweeps and booleans). Most likely IfcReparametrisedCompositeC is also intended for supporting textures so that the U space of the profile prior to extrusion can be controlled.	
Item	Mapped items / reusable definitions	46.3 1247	IfcMappedItem and type object occurrences: Instancing and transformations: one definition, multiple placed occurrences with transforms. In IFC2X3 it was not allowed to re-use the full representation as a whole, later in IFC4 that is possible.	

LEVEL	CONCEPT	PREVALEN	NOTES	RELATED TERMS FROM ISO / OGC (SFA, GEOSPARQL, GML, ...)
Item	(Non-)uniform scaling and mirroring	-	IfcMappedItem is also the only mechanism in IFC that supports scaling and mirroring.	
Item	Composite representations (IfcGeometricSet, IfcCompositeProfile Def, multiple shape representation items)	51.4% (231)	Sets of curves, surfaces, and points combined into a single representation; multi-rep shape representations for one object.	gml:MultiCurve, gml:MultiSurface, gml:CompositeCurve, gml:CompositeSurface
Item	Topological representations for analytical structures	0.3% (760)	IfcEdge IfcVertex IfcFace IfcTopologyRepresentation Explicit topology (vertices, edges, faces) as top-level items; often used in structural/analytical models.	
Item	Point clouds and coverages	0.2% (1)	IfcShapeRepresentation + IfcCartesianPointList3D	
Context	Representation context (IfcGeometricRepresentationContext): identifier, target scale, clearance geometries, subcontexts	-	Defines modeling context: dimensionality, precision, world coordinate system, true north, representation identifiers; may host clearance contexts.	
Context	Tolerance and precision settings	-	IfcGeometricRepresentationC Precision Model-level precision and tolerances that influence how geometry is interpreted and when elements are considered coincident. Only linear tolerance, no angular setting and no local (e.g per-vertex) possibilities.	
Georeferen	Local engineering CRS (IfcGeometricRepresentationContext.WorldCoordinateSystem, true north)	-	Defines the local engineering coordinate system	
Georeferen	Map/projection-based georeferencing incl. vertical reference	12.4% (2)	IfcMapConversion, IfcProjectedCRS Conversion from local model coordinates to a map/projected CRS; explicit link	ISO 19111 referencing by coordinates and transformations; SFA: SRID on geometries.

LEVEL	CONCEPT	PREVALEN	NOTES	RELATED TERMS FROM ISO / OGC (SFA, GEOSPARQL, GML, ...)
			to EPSG or other CRS identifiers; since IFC4/IFC4x3.	
Georeferen	Postal address	38.6% (3)	IfcPostalAddress, Pset_Address	
Georeferen	IfcSite reference coordinates	-	IfcSite has DMS reference coords for latitude and longitude	

12.3. Methodology for testing prevalence.

Testing the prevalence of entities in existing sample models is often criticized because it is hard to find a body of representative IFC models (every tool and vendor cater to their own niche) and prevalence does not equate value. Lastly, per definition, an approach like this is backwards looking and not forward looking. At the same time, the investigation into 3D GeoSPARQL is explicitly use-case driven.

A random sample of 1_000 files (dataset 1) is taken from a collection of models for which it is allowed to publish in aggregate form. For novel concepts such as (IfcLinearPlacement, IfcRelInterferes, IfcAlignmentSegment, ...) a 500 file sample of IFC4.3 files is selected (dataset 2; because the amount of models for older schema versions is larger). It is possible that there are invalid or partial files in this sample, there is no verdict on quality, correctness or applicability of these files. Example files from the specification can have a distorting impact on the sample, also there is no filtering for (near-) duplicates. Some values such as IfcSite.RefLatitude -Longitude are well known to contain placeholder values on older exports such as based on the headquarters of the software vendor.

For performance reasons we don't use an IFC parser for testing the prevalence of entities. Therefore, in theory it is possible that we match fragments of comments or human readable text in the model, but this is given the patterns highly unlikely. We use `ripgrep rg` to query the model as text files on disk. Due to the text-based parsing we cannot fully asses the context in which entities are used — in particular entities like IfcRelAggregates are discussed here in an building-element context, but are also found in the spatial structure and therefore the prevalence is reported higher.

Table 5

QUERY STRING	DATASET INDEX	% OF MODELS	AVG NR INSTANCES WHEN USED
IFCRELAGGREGATES	1	96.9	29

QUERY STRING	DATASET INDEX	% OF MODELS	AVG NR INSTANCES WHEN USED
IFCCARTESIANPOINT	1	99.2	34194
PROFILEDEF	1	76.9	763
IFCANNOTATIONFILLAREA IFCANNOTATIONSURFACE IFCDRAUGHTINGCALLOUT IFCFILLAREASTYLEHATCHING IFCTEXTLITERAL	1	11.6	101
IFCFACETEDBREP	1	43.9	876
IFCADVANCEDBREP	1	6.5	112
IFCTESSELLATEDFACESET IFCTRIANGULATEDFACESET IFCPOLYGONALFACESET	1	12.6	163
IFCTRIANGULATEDIRREGULARNETWORK	2	2	1
IFCEXTRUDEDAREASOLID IFCREVOLVEDAREASOLID IFCFIXEDREFERENCESWEPTAREASOLID IFCSURFACECURVESWEPTAREASOLID IFCDIRECTRIXDERIVEDREFERENCESWEPTAREASOLID	1	75.5	703
IFCSHAPEREPRESENTATION.*Axis	1	47.6	321
IFCBOOLEANRESULT IFCBOOLEANCLIPPINGRESULT	1	30.6	122
IFCBLOCK IFCRECTANGULARPYRAMID IFCRIGHTCIRCULARCONE IFCRIGHTCIRCULARCYLINDER IFCSPHERE	1	1.7	8
IFCCURVEBOUNDEDPLANE IFCCURVEBOUNDEDSURFACE IFCBSPLINESURFACE IFCCYLINDRICALSURFACE IFCPLANE\(IFCSPHERICALSURFACE IFCTOROIDALSURFACE IFCSURFACEOFLINEAREXTRUSION IFCSURFACEOFREVOLUTION	1	31.8	1521
IFCSTYLEDITEM IFCSURFACESTYLE IFCCURVESTYLE	1	68.3	1458
-			
IFCBLOBTEXTURE IFCIMAGETEXTURE IFCPIXELTEXTURE	1	0.8	2
IFCMAPPEDITEM	1	46.3	1247

QUERY STRING	DATASET INDEX	% OF MODELS	AVG NR INSTANCES WHEN USED
-			
IFCGEOMETRICSET IFCCOMPOSITEPROFILEDEF IFCGEOMETRICCURVESET IFCSHAPEREPRESENTATION\(.\\(,	1	51.4	231
IFCTOPOLOGYREPRESENTATION	1	0.3	760
IFCSHAPEREPRESENTATION.*Point	1	0.2	1
-			
-			
-			
IFCPROJECTEDCRS IFCGEOGRAPHICCRS IFCMAPCONVERSION IFCRIGIDOPERATION	2	12.4	2
IFCPOSTALADDRESS Pset_Address	1	38.6	3
-			

12.4. 5.3 Vanilla 3D geometry handling in the Semantic Web

This section describes in what other ways 3D geometry is currently handled in the Semantic Web, for example in BOT ontology, OMG and FOG ontologies, and few more.

12.5. 5.4 Concluding overview of requirements for 3D geometry in the semantic web

A concluding summary with a list of requirements to be taken into account for future development in different places and organisations.



13

BENEFICIARIES AND BENEFITS

This section describes the beneficiaries and benefits of representing data, including geospatial data, using semantic and graph technologies. Furthermore, a collection of use cases demonstrate how semantic and graph technologies are used together with spatial data to tackle real world problems.

13.1. Semantic-functional value of 3D-GeoSPARQL

Value for data reasoning, formal description, and semantic interoperability

13.1.1. Meaningful creation and querying of 3D geometry

Whether manually or programmatically defining 3D geometries in BIM, GEO or CG environments, the GeoSPARQL standard could provide or connect to vocabulary to represent 3D structures. A 3D cube, for example, can be represented in multiple ways:

- As a set of bounding points, lines, and faces;
- As a base surface with an extrude function;
- As an 3D geometric primitive (e.g., the concept of a ‘cube’ as a spatial figure).

Once GeoSPARQL offers the means to define and link 3D geometries in these different ways, it will allow both humans and machines to interpret and interact with the data effectively.

13.1.2. Meaningful creation and querying of 3D topological relationships

Is “3D Geometry A” inside, touching, overlapping, intersecting or above “3D Geometry B”?

Being able to describe and use such topological relationships is crucial for conducting spatial analyses, formulating rules, and deriving knowledge from different heterogeneous datasets.

Beyond describing topological relationships, one should be able to query them:

What is the spatial relationship between 3D Geometry A and 3D Geometry B?

Such queries should return the appropriate topological result (e.g., “intersects”), which is essential for use cases like clash detection in design validation

13.1.3. Derivation of geometric and topological knowledge

From explicitly modeled geometric and topological data, one should be able to infer implicit knowledge.

For instance, if 3D Geometry A is 3D inside 3D Geometry B, then we can infer that:

A does not “touch” B, and A does not “overlap” with B,

even if these facts are not explicitly stated.

For entailment an application that is compliant with the geo-spatial 3D specification would be able to answer queries like: “Does this tree have a 3D geometry?”

If the tree is defined using a MultiSolid geometry, then the answer should be “yes,” even if this is not explicitly declared as such.

13.2. Value for user experience, visualization, and application use of 3D-GeoSPARQL

13.2.1. Visualization

There is a growing need to visualize how 3D data in BIM and GIS relate to one another. Stakeholders want to see BIM and/or 3D Geo data of a newly planned structure visualized within the 3D Geo and/or BIM context of the existing digital city. A shared vocabulary that can present both domains in an integrated way supports this goal.

13.2.2. Calculation and analysis

Semantic modelled 3D-model enables powerful computation and analysis. Analytical results can be displayed in dashboards operating within the GeoBIM domain. This creates a bridge between asset management (typically GIS-oriented) and project management (typically BIM-oriented), allowing for cross-domain collaboration and decision-making.

13.2.3. Spatial querying

Spatial querying is a key function within a federated GeoBIM Network. Questions such as “What material is located in this area?” or “Where is space available for new cables and pipelines?” are examples of 3D spatial queries that can be answered when GeoSPARQL 3D is functioning effectively across systems and semantics.

13.2.4. Machine control

A shared semantic GeoBIM Network also supports machine control. The GIS domain typically describes the existing situation, while the BIM domain describes the intended or future situation. This combination can be used to automatically instruct and guide machines in the built environment.

13.2.5. Modeling, simulation and planning

A network of datasets with 3D-data should support the modeling, simulation and planning of future changes to the built environment. Users — whether human or machine — can use the network and GeoSPARQL 3D to propose and model modifications in either BIM or GIS formats. A well-functioning 3D semantic GeoBIM Network enables this kind of forward-looking spatial planning and design.

13.3. Added value of the 3D-GeoSPARQL for Organisations

13.3.1. Brings together different domains (GIS, BIM, CG)

GeoSPARQL 3D can help in bringing together different domains that work with 3D information.

13.3.2. Enables extension to vocabularies from other domains

The vocabulary can have the possibility to extend to vocabulaires with 3D geometry or topology from other domains, for example the Building Ontology Topology (BOT) or RELOC Ontology. Multi-domain models with georeferenced 3D and 2D city models could be also linked by CityRDF ontology based on CityGML 3.0 standard. GeoSPARQL 3D would be important enabler for data retrieval, update and analysis for such models. In addition to ontologies, it can also establish relationships to 3D file formats from other domains, such as glTF.

13.3.3. Facilitates better interoperability between knowledge domains with spatial components (geography, aerospace, architecture, product design, (bio)chemistry, IoT, ...)

A unified geometric language can help by migrating ideas and methods across fields, accelerating innovation. For example a parametric design algorithm, rule, or IIm making use of geometry and topology originated in aerospace, can be re-used in the architecture domain.

14

ANNEX N: ALIGNMENT GEOSPARQL AND OPEN SOURCE RELATIONAL DATABASES WITH GEOSPATIAL SUPPORT .

ANNEX N: ALIGNMENT GEOSPARQL AND OPEN SOURCE RELATIONAL DATABASES WITH GEOSPATIAL SUPPORT .

Based on:

GeoSPARQL: PostGIS: https://postgis.net/docs/PostGIS_Special_Functions_Index.html#PostGIS_3D_Functions (PostGIS Functions that support 3D) H2GIS: <https://h2gis.readthedocs.io/en/latest/index.html> (paragraph Geometry 3D/4D functions) Spatialite: <https://www.gaia-gis.it/gaia-sins/spatialite-sql-latest.html>



15

ALIGNMENT: GEOSPARQL AND OPEN SOURCE RELATIONAL DATABASES WITH GEOSPATIAL SUPPORT

ALIGNMENT: GEOSPARQL AND OPEN SOURCE RELATIONAL DATABASES WITH GEOSPATIAL SUPPORT

15.1. Formatting functions

Table 6

GeoSPARQL	PostGIS	H2GIS	Spatialite
geof:asGeoJSON	<u>st_asgeojson</u>	<u>ST_AsGeoJson</u>	asgeojson
geof:asGML	<u>st_asgml</u>	<u>st_asgml</u>	asgml
geof:asKML	<u>st_askml</u>	<u>ST_AsKML</u>	askml
geof:asWKT		st_aswkt	aswkt
geof:asEWKB	<u>st_asewkb</u>	<u>AsEWKB</u>	asewkb
	<u>st_ashexewkb</u>		
	<u>ST_AsX3D</u>		asx3d
	link:st_asewkt		asewkt
	st_assvg		assvg
	st_astext	st_astext	astext
			astwkb
	<u>st_asbinary</u>	st_asbinary	asbinary
geof:asDGGs			
geof:asGeocode			

	asfgf
	asgpb
	asencodedpolyline
	atm_astext
	gcp_astext
	proj_asprojstring
	proj_aswkt

15.2. Convert geometries

Table 7

GeoSPARQL	PostGIS	H2GIS	Spatialite
	<u>st_Force2d</u>	<u>st_force2d</u>	
	<u>st_ForceCurve</u>		
	<u>st_force_3d</u>	<u>st_force3d</u>	
	<u>st_force_3dm</u>	<u>st_force3dm</u>	
	<u>st_force_3dz</u>		
	<u>st_force_4d</u>	<u>st_Force4D</u>	
	<u>st_force_collection</u>		
			forceasnull
	<u>st_forcerhr</u>		
	<u>CG_ForceLHR</u>		forcelhr

<u>st_forcerSFS</u>	
<u>ForcePolygonCCW</u>	forcepolygonccw
<u>ForcePolygonCW</u>	forcepolygoncw
	normalizelonlat
	project
	spatnetfromgeom
	spatnetfromtgeo
	sridfromauthcrs
<u>st_shift_longitude</u>	st_shift_longitude
st_wkbtoql	st_wkbtoql
st_wkttosql	st_wkttosql
	tinypointencode
	togars
	uncompressgeometry
	xb_getgeometry
	st_projectpoint
	st_tomultipoint
	st_tomultisegments
<u>st_curvetoline</u>	
st_geogfromtext	
st_geogfromwkb	
st_geographyfromtext	

st_geometryfromtext
st_gmltosql
st_linefromwkb
st_linestringfromwkb
st_linetocurve

15.3. Create geometries

Table 8

	st_extrude	<u>st_extrude</u>	
	<u>CG_Extrude</u>		
		<u>st_geometryshadow</u>	
geof:boundary	<u>st_boundary</u>	st_boundary	boundary
geof:boundingCircle		st_boundingcircle	
	<u>st_BoundingDiagonal</u>		
	st_minimumboundingcircle		
geof:concaveHull			concavehull
geof:convexHull	<u>st_convexhull</u>	st_convexhull	convexhull
	<u>CG_3DConvexHull</u>		
geof:difference	<u>st_difference</u>	st_difference	difference
	<u>CG_3DDifference</u>		
geof:envelope	st_envelope	st_envelope	envelope

geof:symDifference	<u>st_symdifference</u>	st_symdifference	symdifference
geof:union	<u>st_union</u>	st_union	gunion
	<u>CG_3DUnion</u>		
geof:patchN	<u>st_PatchN</u>		
	<u>CG_MakeSolid</u>		
	<u>ST_LineSubstring</u>		
			atm_create
			atm_createrotate
			atm_createscale
			atm_createtranslate
			atm_createxroll
	st_bdmpolyfromtext		bdmpolyfromtext
			bdmpolyfromwkb
	st_bdpolyfromtext		bdpolyfromtext
			bdpolyfromwkb
			blobfromfile
	st_buildarea		buildarea
			buildcirclebr
			buildmbr
			castautomagic
			casttoblob
			casttodouble

		casttogeometycollection
		casttointeger
		casttolinestring
		casttomulti
	st_tomultiline	casttomultilinestring
		casttomultipoint
		casttomultipolygon
		casttopoint
		casttopolygon
		casttosingle
		casttotext
		casttoxy
		casttoxym
		casttoxyz
		casttoxyzm
<u>st_collect</u>	st_collect	collect
st_collectionextract	st_collectionextract	collectionextract
		createnetwork
		createroutingnodes
		createtopogeo
		createtopology
		dissolvepoints

		dissolvesegments
		elementarygeometries
		extractmultilinestring
		extractmultipoint
		extractmultipolygon
		geomcollfromtext
		geomcollfromwkb
		geometrycollectionfromtext
		geometrycollectionfromwkb
		geometrypointencode
<u>st_geomfromewkb</u>		geomfromewkb
<u>st_geomfromewkt</u>		geomfromewkt
		geomfromexifgpsblob
		geomfromfgf
<u>st_GeomFromGeoJSON</u>		geomfromgeojson
<u>st_geomfromgml</u>	st_geomfromgml	geomfromgml
		geomfromgpb
<u>st_geomfromkml</u>		geomfromkml
st_geomfromtext	st_geomfromtext	geomfromtext
st_geomfromwkb	st_geomfromwkb	geomfromwkb
		geosconcavehull
		gpkgmakepoint

gpkgmakepointm		
gpkgmakepointz		
gpkgmakepointzm		
gunion		
hexagonalgrid		
hilbertcode		
linefromencodedpolyline		
st_linefromtext	st_linefromtext	"linefromtext linestringfromtext"
"linefromwkb linestringfromwkb"		
linesfromrings		
linestringfromtext		
linestringfromwkb		
makearc		
makecircle		
makecircularsector		
makecircularstripe		
st_makeellipse		makeellipse
makeellipticarc		
makeellipticsector		
<u>st_makeline</u>	st_makeline	makeline
<u>st_makepoint</u>	st_makepoint	makepoint
st_makepointm	makepointm	

		makepointz
		makepointzm
<u>st_makepolygon</u>	st_makepolygon	makepolygon
		mlinefromtext
		mlinefromwkb
		mpointfromtext
st_mpointfromtext	st_mpointfromtext	"mpointfromtext multipointfromtext"
		"mpointfromwkb multipointfromwkb"
		mpolyfromtext
		mpolyfromwkb
		multilinestringfromtext
		multilinestringfromwkb
		multipolygonfromtext
		multipolygonfromwkb
		orientedenvelope
st_pointfromtext	st_pointfromtext	pointfromtext
<u>st_pointfromwkb</u>	st_pointfromwkb	pointfromwkb
<u>st_pointonsurface</u>	st_pointonsurface	pointonsurface
		polyfromtext
		polyfromwkb
		polygonfromtext
		polygonfromwkb

st_polygonize	st_polygonize	polygonize
		squaregrid
<u>st_point</u>	st_point	st_point
		topogeo_polygonize
		triangulargrid
		voronojdiagram
	st_isovist	
	st_linefromwkb	
	st_makeenvelope	
	st_makegrid	
	st_makegridpoints	
	st_minimumrectangle	
st_mlinefromtext	st_mlinefromtext	
st_mpolyfromtext	st_mpolyfromtext	
	st_octogonalenvelope	
st_polygonfromtext	st_polyfromtext	
	st_polyfromwkb	
	st_ringbuffer	
	st_ringsidebuffer	
	st_sidebuffer	
	<u>st_tessellate</u>	
<u>CG_Tessellate</u>		

<u>st_linefrommultipoint</u>
st_makebox2d
st_makebox3d
st_makeenvelope
st_multi
<u>st_polygon</u>

15.4. Edit geometries

Table 9

GeoSPARQL	PostGIS	H2GIS	Spatialite
		<u>st_addz</u>	
	<u>AddGeometryColumn</u>		addgeometrycolumn
	<u>st_addmeasure</u>		addmeasure
	<u>st_addpoint</u>	st_addpoint	addpoint
			addedgemodface
			addedgenewfaces
			addfdogeometrycolumn
	<u>DropGeometryColumn</u>		
			addisoedge
			addisonetnode
			addisonode

		addlink
		addtemporarygeometrycolumn
		xb_addfileid
		xb_addparentid
	<u>st_interpolate3dline</u>	
<u>ST_InterpolatePoint</u>		interpolatepoint
<u>ST_LineInterpolatePoint</u>		
<u>ST_LineInterpolatePoints</u>		
<u>ST_3DLineInterpolatePoint</u>		
		line_interpolate_equidistant_points
st_line_interpolate_point		line_interpolate_point
		trajectoryinterpolatepoint
	<u>st_multiply</u>	
<u>st_reverse</u>	st_reverse	reverse
	<u>st_reverse3dline</u>	
<u>UpdateGeometrySRID</u>		
		storedvar_updatetitle
		storedvar_updatevalue
		se_updatevectorcoverageextent
		storedproc_updatesqlbody
		storedproc_updatetitle
		topogeo_updateseeds

			toponet_updateseeds
			updatelayerstatistics
			updatemetacatalogstatistics
			<u>st_zupdatelineextremities</u>
			<u>st_updatez</u>
geof:buffer	st_buffer	st_buffer	buffer
geof:transform	st_transform	st_transform	transform
geof:metricBuffer			
geof:transform CRS84			
			<u>CG_ChaikinWmoothing</u>
			<u>ST_Scroll</u>
			<u>ST_WrapX</u>
			atm_invert
			atm_multiply
			atm_rotate
			atm_scale
			atm_transform
			atm_translate
			atm_xroll
			atm_yroll
			blobtofile
			bufferoptions_reset

		bufferoptions_setendcapstyle
		bufferoptions_setjoinstyle
		bufferoptions_setmitrelimit
		bufferoptions_setquadrantsegments
		changeedgegeom
		changelinkgeom
		compressgeometry
		discardfdogeometrycolumn
		discardgeometrycolumn
		drapeline
		drapelineexceptions
		dropnetwork
		droptopology
		dropvirtualgeometry
		ensureclosedrings
st_expand	st_expand	expand
		gcp_transform
		gcp2atm
		geosdensify
		geosmakevalid
		gpkginsertepsgsrid
		inittopogeo

		inittoponet
st_line_substring		line_substring
st_linemerge	st_linemerge	linemerge
		linescutatnodes
		loginetfromtgeo
<u>st_makevalid</u>	st_makevalid	makevalid
		makevaliddiscarded
		modedgeheal
		modedgesplit
		modgeolinksplit
		modlinkheal
		modloglinksplit
		moveisonetnode
		moveisonode
		newedgeheal
		newedgessplit
		newgeolinksplit
		newlinkheal
		newloglinksplit
		offestcurve
		reduceprecision
		reflectcoordinates

	reflectcoords
	remedgemodface
	remedgenewface
	remisoedge
	remisonetnode
	remisonode
	removelink
<u>st_removepoint</u>	removepoint
	ringscutatnodes
	rotatecoordinates
	rotatecoords
	sanitizegeometry
	scalecoordinates
	scalecoords
<u>st_segmentize</u>	segmentize
	setdecimalprecision
	setendpoint
	setmultiplepoints
<u>st_setpoint</u>	setpoint
	setstartpoint
	shiftcoordinates
	shiftcoords

st_simplify	st_simplify	simplify
<u>CG Simplify</u>		
st_simplifypreservetopology	st_simplifypreservetopology	simplifypreservetopology
		singlesidedbuffer
st_snap		snap
		snapandsplit
<u>st_snaptogrid</u>		snaptogrid
st_split		split
		splitleft
		splitright
		st_cutter
<u>st_node</u>	st_node	st_node
		st_subdivide
<u>st_translate</u>	st_translate	st_translate
<u>CG 3DTranslate</u>		
		swapcoordinates
<u>swapordinates</u>		
		swapcoords
		transformxy
		transformxyz
<u>st_UnaryUnion</u>	unaryunion	
st_densify		

	st_explode
<u>ST_FlipCoordinates</u>	st_flipcoordinates
	st_normalize
	st_precisionreducer
	st_ removeduplicatedcoordinates
	st_removeholes
	st_removepoints
<u>ST_RemoveRepeated Points</u>	st_removeRepeatedpoints
<u>st_rotate</u>	st_rotate
<u>CG_3DRotate</u>	
<u>st_rotatex</u>	
<u>CG_RotateX</u>	
<u>CG_RotateY</u>	
<u>CG_RotateZ</u>	
<u>st_rotatey</u>	
<u>st_rotatez</u>	
<u>st_scale</u>	st_scale
<u>CG_3DScale</u>	
<u>CG_3DScaleAroundCenter</u>	
	st_split
<u>st_affine</u>	
<u>st_dump</u>	

<u>st DumpPoints</u>
<u>st_dumpings</u>
<u>st_dumpsegments</u>
<u>st_memunion</u>
<u>st_transscale</u>

15.5. Topological Relations

Table 10

GeoSPARQL	PostGIS	H2GIS	Spatialite
geof:sfContains	st_contains	st_contains	contains
geof:ehCoveredBy	st_coveredby		coveredby
geof:ehCovers	st_covers	st_covers	covers
geof:sfCrosses	st_crosses	st_crosses	crosses
geof:sfDisjoint	st_disjoint	st_disjoint	mbrdisjoint
geof:sfEquals	st_equals	st_equals	equals
<u>TG_Equals</u>			
geof:intersection	<u>st_intersection</u>	st_intersection	intersection
<u>CG_3DIntersection</u>			
geof:sfIntersects	st_intersects	st_intersects	intersects
<u>tg_intersects</u>			
<u>ST_3DIntersects</u>			

geof:sfOverlaps	st_overlaps	st_overlaps	overlaps
	<u>&&&</u>		
	<u>&&&(geometry,gidx)</u>		
	<u>&&&(gidx, geometry)</u>		
	<u>&&&(gidx,gidx)</u>		
geof:sfTouches	st_touches	st_touches	touches
geof:sfWithin	st_within	st_within	within
	st_dfullywithin		
	<u>st_3DDFullyWithin</u>		
geof:ehContains			
geof:ehDisjoint			
geof:ehEquals			
geof:ehInside			
geof:ehMeet			
geof:ehOverlap			
geof:rcc8dc			
geof:rcc8ec			
geof:rcc8eq			
geof:rcc8ntpp			
geof:rcc8ntppi			
geof:rcc8po			
geof:rcc8tpp			

geof:rcc8tppi			
geof:relate	st_relate	st_relate	relate
<u>st_CPAWithin</u>			
buildmbrfilter			
st_closestpoint		st_closestpoint	closestpoint
<u>ST_ClosestPointOfApproach</u>			
<u>ST_3DClosestPoint</u>			
		st_envelopesintersect	envelopesintersects
filtermbrcontains			
filtermbrintersects			
filtermbrwithin			
garsmbr			
getfacebypoint			
getfaceedges			
getfacegeometry			
getlinkbypoint			
getnetnodebypoint			
getnodebypoint			
ptdistwithin			
relatemark			
st_setsrid		st_setsrid	setsrid
sharedpaths			

	validatetopogeo
	validlogicalnet
	validspatialnet
st_dwithin	st_dwithin
	st_lineintersector
	st_orderingequals
@	
&&	
&<	
&<	
&>	
<<	
<<	
=	
>>	
&>	
>>	
~	
~=	
st_containsproperly	
st_linecrossingdirection	
st_orderingequals	

15.6. Measure

Table 11

GeoSPARQL	PostGIS	H2GIS	Spatialite
geof:area	st_area	st_area	area
	<u>CG_3DArea</u>	<u>st_3darea</u>	
geof:perimeter	st_perimeter	st_perimeter	perimeter
	st_perimeter2d		
	<u>st_perimeter3d</u>	<u>st_3dperimeter</u>	
geof:metricPerimeter			
geof:length	st_length	st_length	length
	<u>ST_3DLength</u>	<u>ST_3DLength</u>	3dlength
geof:metricArea			
geof:metricLength			
			geodesic length
			geodesicarcarea
			geodesicarchheight
			geodesicarclength
			geodesiccentralangle
			geodesicchordlength

			great circle length
			linestringavgsegmentlength
<u>st_length_spheroid</u>			
st_length2d			
st_length2d_spheroid			
st_length3d_spheroid			
			linestringminsegmentlength
<u>st_svf</u>			
<u>st_sunposition</u>			
geof:distance	st_distance	st_distance	distance
<u>st_distanceCPA</u>			
geof:metricDistance			
<u>ST_3DDistance</u>		3ddistance	
<u>CG_Volume</u>			
<u>ST_GeometricMedian</u>			
			abs
			acos
			asin
			atan
			atan2
st_azimuth		st_azimuth	azimuth
			ceil

		ceiling
		centimeter
		circularity
		cos
		cot
		curvosityindex
		dd to dms
		decimeter
		degrees
		distancewithin
		dms to dd
		downhillheight
		exp
st_extent	st_extent	extent
<u>ST_3DExtent</u>		
		floor
		frechetdistance
		gcp_compute
		geosmaximuminscribedcircle
		geosminimumboundingcenter
		geosminimumboundingcircle
		geosminimumboundingradius

		geosminimumclearance
		geosminimumclearanceline
		geosminimumrotatedrectangle
		geosminimumwidth
		getlayerextent
		getvirtualtableextent
st_hausdorffdistance		hausdorffdistance
st_line_locate_point		line_locate_point
st_locate_along_measure		locatealongmeasure
st_locate_between_measures		locatebetweenmeasures
st_maxdistance	st_maxdistance	maxdistance
<u>3DMaxDistance</u>		3dmaxdistance
st_shortestline	st_shortestline	shortestline
<u>st_3dshortestline</u>		
		updownheight
		uphillheight
		st_closestcoordinate
		st_furthestcoordinate
		st_locatealong
st_longestline	st_longestline	
<u>st_3DLongestLine</u>		
st_distance_sphere		

15.7. Count

Table 12

GeoSPARQL	PostGIS	H2GIS	Spatialite
geof:numGeometries	<u>st_numgeometries</u>	st_numgeometries	numgeometries
geof:numPoints	st_numpoints	st_numpoints	numpoints
	<u>st_numpatches</u>		
geof:numInteriorRing			
geof:numPatches			
	<u>ST_NumCurves</u>		
	<u>st_npoints</u>	st_npoints	npoints
	<u>st_nrings</u>		nrings
	st_numinteriorring	st_numinteriorring	numinteriorring
	st_numinteriorrings	st_numinteriorrings	numinteriorrings
	<u>st_locatebetweenelevations</u>		

15.8. Properties

Table 13

GeoSPARQL	PostGIS	H2GIS	Spatialite
-----------	---------	-------	------------

		<u>st_is3d</u>	is3d
geof:x	<u>st_x</u>	st_x	x
geof:maxX	<u>st_xmax</u>	st_xmax	
geof:minX	<u>st_xmin</u>	st_xmin	
geof:y	<u>st_y</u>	st_y	y
geof:maxY	<u>st_ymax</u>	st_ymax	
geof:minY	<u>st_ymin</u>	st_ymin	
geof:z	<u>st_z</u>	<u>st_z</u>	z
geof:maxZ	<u>st_zmax</u>	<u>st_zmax</u>	maxz
geof:minZ	<u>st_zmin</u>	<u>st_zmin</u>	minz
geof:m	<u>st_m</u>	<u>ST_M</u>	m
geof:centroid	st_centroid	st_centroid	centroid
geof:geometryN	st_geometryn	st_geometryn	geometryn
geof:getSRID	st_srid	st_srid	srid
	geof:isEmpty	st_isempty	st_isempty
isempty	geof:isSimple	<u>st_issimple</u>	st_issimple
issimple	geof:endPoint	<u>st_endpoint</u>	st_endpoint
endpoint	geof:exteriorRing	<u>st_exteriorring</u>	st_exteriorring
exteriorring	geof:interiorRingN	<u>st_interiorringn</u>	st_interiorringn
interiorringn	geof:isRing	st_ising	st_ising
ising	geof:pointN	<u>st_pointn</u>	st_pointn
pointn	geof:startPoint	<u>st_startpoint</u>	st_startpoint

startpoint	geof:easting		
	geof:northing		
	geof:isClosed	st_isclosed	st_isclosed
isclosed		CG_IsPlanar	
		CG_IsSolid	
		ST_IsCollection	
atm_determinant			
atm_isinvertible			
atm_isvalid			
bufferoptions_getendcapstyle			
bufferoptions_getjoinstyle			
bufferoptions_getmitrelimit			
bufferoptions_getquadrantsegments			
checkwithoutrowid			
checkspatialindex			
checkspatialmetadata		st_coorddim	st_coorddim
coorddimension		st_dimension	st_dimension
dimension			
gcp_isvalid		st_geohash	
geohash		Geometrytype	
geometrytype			

getdecimalprecision
getgpkgamphibiousmode
getgpkgmode
getisometadataid
getpointindex
gpkg_isassignable
gpkggetimagetype
gpkggetnormalrow
gpkggetnormalzoom
haspsg
hasfreexl
hasgcp
hasgeocallbacks
hasgeopackage
hasgeos
hasgeos3100
hasgeos3110
hasgeosadvanced
hasgeosonlyreentrant
hasgeosreentrant
hasgeostrunk
hasiconv

hasknn			
haslibxml2			
hasmathsql			
hasminizip			
hasproj			
hasproj6			
hasrouting			
hasrttopo			
hastopology			
isgeometryblob			
iscompressedgeometryblob			
ismeasured	ST_IsPolygonCCW		
ispolygonccw	ST_IsPolygonCW		
ispolygoncw	st_isvalid	st_isvalid	
isvalid	st_IsValidTrajectory		
		st_isvaliddetail	
isvaliddetail	st_isvalidreason	st_isvalidreason	
isvalidreason			
isvalidtrajectory			
maxm			
mbrmaxx			
mbrmaxy			

mbrminx	
mbrminy	
minm	<u>st_ndims</u>
ndims	
selfintersections	st_compactnessratio
	st_geometrytype
	st_geometrytypecode
	st_holes
	st_isrectangle
	<u>box3d</u>
	<u>st_curveN</u>
	st_estimated_extent
	st_extent3d
	<u>st_hasarc</u>
	<u>st_hasM</u>
	<u>st_hasZ</u>
	st_summary
	<u>st_zmflag</u>

15.9. Topography

Table 14

GeoSPARQL	PostGIS	H2GIS	Spatialite
		<u>st_drape</u>	
		<u>st_triangleaspect</u>	
		<u>st_trianglecontouring</u>	
		<u>st_triangledirection</u>	
		<u>st_triangleslope</u>	
		<u>st_voronoi</u>	
	<u>CG_ApproximateMedialAxis</u>		
	<u>CG_Orientation</u>		
	<u>CG_StraightSkeleton</u>		
	<u>CG_Visibility</u>		
			geoslargestemptycircle
	<u>st_mem_size</u>		

15.10. Triangulation

Table 15

GeoSPARQL	PostGIS	H2GIS	Spatialite
		<u>st_constraineddelaunay</u>	constraineddelaunaytriangulation
	<u>st_DelaunayTriangles</u>		
	<u>CG_ConstrainedDelaunayTriangles</u>		
			delaunaytriangulation

[st_delaunay](#)

[CG_AlphaWrapping](#)

15.11. Export/export

Table 16

GeoSPARQL	PostGIS	H2GIS	Spatialite
			exportdbf
			exportdxs
			exportgeojson2
			exportkml
			exportshp
			importdbf
			importdxs
			importdxsfromdir
			importgeojson
			importshp
			importwfs
			importxls
			importzipdbf
			importzipshp
			initfdospatialmetadata

	initspatialmetadata
	initspatialmetadatafull
	insertepsgsrid
	topogeo_exporttopolayer
	topogeo_fromgeotable
	topogeo_fromgeotableext
	topogeo_fromgeotablenoface
	topogeo_fromgeotablenofaceext
	topogeo_togeotable
	topogeo_togeotablegeneralize
	toponet_fromgeotable
	toponet_togeotable
	toponet_togeotablegeneralize
find_srid	

15.12. Other

Table 17

GeoSPARQL	PostGIS	H2GIS	Spatialite
			autofdostart
			autofdostop
			autogpkgstart

autogpkgstop
check_strict_sql_quoting
checkduplicaterows
checkgeopackagemetadata
checkshadowedrowid
clonetable
countunsafetriggers
createclonedtable
createisometadatatables
creatembrcache
createmetacatalogtables
createmissingrasterlite2columns
createmissingsystemtables
createrrastercoveragestable
createrrouting
createrrouting_getlasterror
createspatialindex
createtestingtables
createtemporaryspatialindex
createtopotables
createuuid
createvectorcoveragestable

decodeurl
dirnamefrompath
disablepkgamphibiousmode
disablepkgmode
disablepause
disablespatialindex
disabletinypoint
droptable
enablepkgamphibiousmode
enablepkgmode
enablepause
enabletinypoint
encodeurl
eval
fileextfrompath
filenamefrompath
freexl_version
fullfilenamefrompat
geos_version
getcuttermessage
getdbobjectscope
getlastnetworkexception

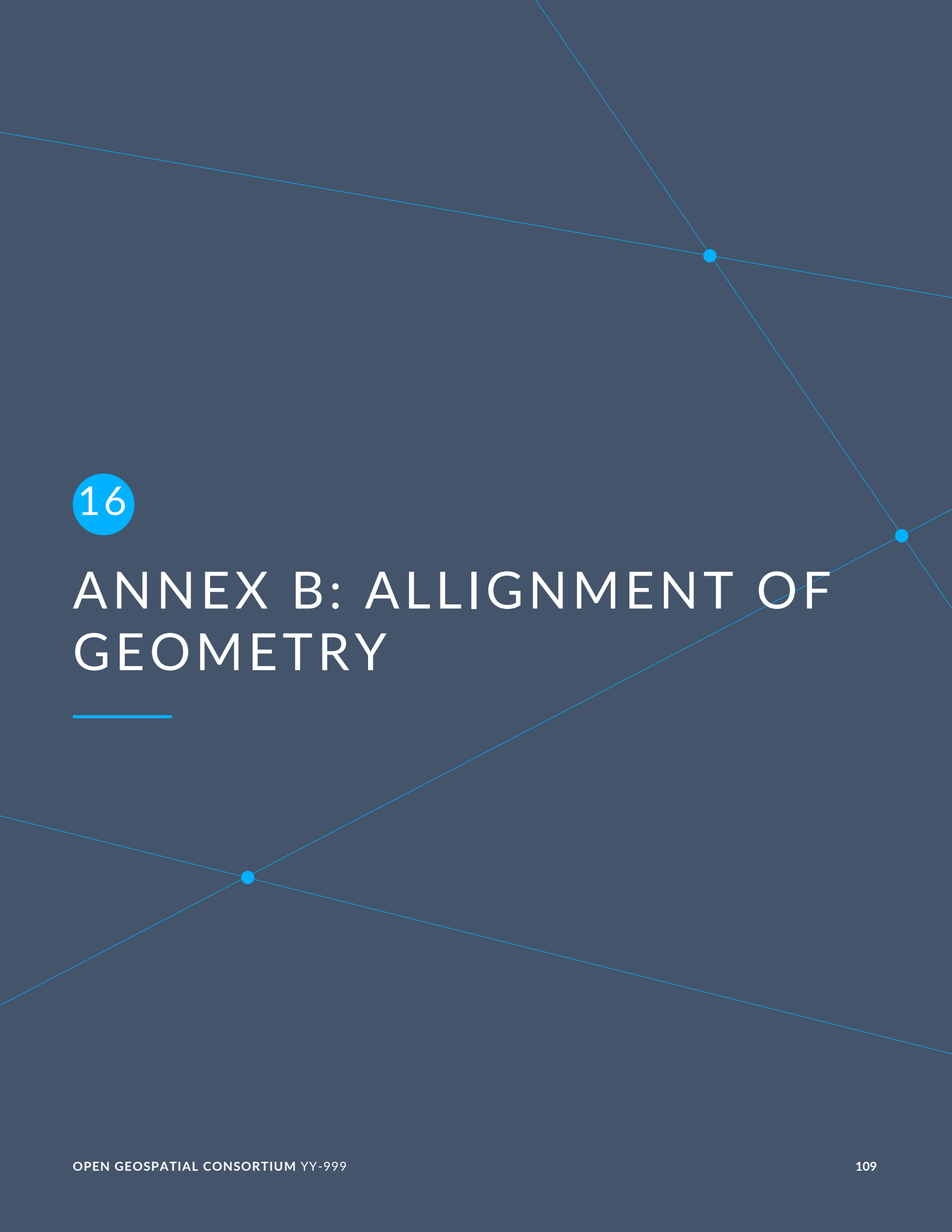
getlasttopologyexception
getmimetype
gpkgcreatebasetables
gpkgcreatetilestable
gpkgcreatetileszoomlevel
indian chain
indian foot
indian yard
international chain
international fathom
international foot
international inch
international link
international nautical mile
international statute mile
international yard
invalidatelayerstatistics
isdecimalnumber
isexifblob
isexifgpsblob
isgifblob
isinteger

isjp2blob
isjpegblob
islowascii
isnumber
ispauseenabled
ispdfblob
ispngblob
istiffblob
istinypointblob
istinypointenabled
isvalidgpb
iswebpblob
iszipblob
kilometer
libxml2_version
ln
log
log10
log2
makestringlist
md5checksum
md5totalchecksum

millimeter
pause
pi
pow
power
proj_getdatabasepath
proj_guesssridfromshp
proj_guesssridfromwkt
proj_guesssridfromzipshp
proj_setdatabasepath
proj_version
radians
rebuildgeometrytriggers
recoverfdogeometrycolumn
recovergeometrycolumn
recoverspatialindex
recreateisometarefstriggers
recreaterrastercoveragestriggers
recreatestylingtriggers
recreatetopotriggers
recreatevectorcoveragestriggers
registerdatalicense

registerisometadata
registervirtualgeometry
removeduplicaterows
removeextraspaces
renamecolumn
renamedatalicense
renametable
rl2_mapconfigurationabstractn
rl2_mapconfigurationnamen
rl2_mapconfigurationtitlen
rl2_nummapconfigurations
rl2_registermapconfiguration
rl2_reloadmapconfiguration
rl2_unregistermapconfiguration
rttopo_version
sequence_curval
sequence_lastval
sequence_nextval
sequence_setval
setdatalicenseurl
sign
sin

	spatialite_target_cpu
	spatialite_version
	sqrt
	stddev_pop
	stddev_samp
	tan
	u.s. chain
	u.s. foot
	u.s. inch
	u.s. statute mile
	u.s. yard
	unregisterdatalicense
	upgradegeometrytriggers
	var_pop
	var_samp
st_accum	st_accum
	st_googlemaplink
	st_osmmaplink

The background is a solid dark blue-grey color. It features several thin, light blue lines that intersect at various points. Three of these intersection points are marked with small, solid blue dots. The lines and dots create a geometric, network-like pattern across the page.

16

ANNEX B: ALIGNMENT OF GEOMETRY

Overview of ISO 19107 GML IFC STEP OpenCascade WKT X3D Geometries

Dit overzicht van geometriën laat zien welke geometrieën dicht bij elkaar zitten. Het is geen 10% match

Points:

Table 18

ISO 19107	GML	IFC	STEP	OpenCascade	WKT
		<u>IfcCartesianPoint</u>	<u>cartesian_point</u>	<u>Geom2d_Cartesian Point</u> <u>Geom_Cartesian Point</u>	
<u>GM Point</u>	<u>gml: Point</u>			<u>Geom2d Point</u> <u>Geom Point</u>	<u>POINT (x y (z (m)))</u>
			<u>cylindrical_point</u>		
			<u>spherical_point</u>		
			<u>polar_point</u>		
		<u>IfcPointByDistance Expression</u>			
		<u>IfcPointOnCurve</u>	<u>point_on_curve</u>		
		<u>IfcPointOnSurface</u>	<u>point_on_surface</u>		
			<u>point_in_volume</u>		
			<u>point_replica</u>		
			<u>degenerate_pcurve</u>		
			<u>evaluated_degenerate_pcurve</u>		

Lines and Curves:

Table 19

ISO 19107	GML	IFC	STE OpenCascade	WKT
<u>GM_Curve</u>	<u>gml:Curve</u>			
<u>GM_Orientable Curve</u>	<u>gml:Orientable Curve</u>			
GM_Arc	<u>gml:Arc</u>			<u>CIRCULARSTRING</u> (x y (z)(m))
GM_Arc ByBulge	<u>gml:ArcByBulge</u>			
GM_ArcString	<u>gml:ArcString</u>			
GM_ArcString ByBulge	<u>gml:ArcString ByBulge</u>			
	<u>gml:Circle ByCenterPoint</u>			
	<u>gml:CubicSpline</u>			
	<u>gml:Knot</u>			
	<u>gml:Affine Placement</u>			
	<u>gml:Abstract Surface</u>			
	<u>gml:Polygon</u>			<u>POLYGON(x y (z) (m))</u>
GM_Polygon	<u>gml:Polygonpatch</u>			
<u>GM_Ring</u>	<u>gml:Ring</u>			
	<u>gml:AbstractRing</u>			
	<u>gml:LinearRing</u>			
	<u>gml:Surface</u>			
	<u>gml:Patches</u>			
	<u>gml:AbstractRing</u>			
<u>GM_Orientable Surface</u>	<u>gml:Orientable Surface</u>			

	gml:Bspline	IfcBSplineCurveWithKnots	B_splir Curve Geom2d_BSpline Curve Geom_BSpline with Curve knot
		IfcRationalBSplineCurveWithKnots	
GM_Bezier	gml:Bezier		Geom2D_Bezier bezi Curve curv Geom_Bezier Curve
			ratic b_ splir curv
			loca b_ splir
			unifi curv
			quas unifi curv
			loca refir splir curv
			ratic loca refir splir curv
			Geom2d_Trimmed trim Curve curv Geom_Trimmed Curve
		IfcCompositeCurveOnSurface	
		IfcBoundaryCurve	bou curv
		IfcOuterBoundaryCurve	oute bou curv

<u>IfcGradientCurve</u>			
<u>IfcSegmentedReferenceCurve</u>			
<u>IfcIndexedPolyCurve</u>			
<u>GM_LineString</u>	<u>gml:LineString</u> Segment		LINESTRING(x y (z)(m))
	<u>gml:LineString</u>	<u>IfcPolyline</u>	<u>poly</u> <u>TopoDS_Wire</u>
<u>IfcTrimmedCurve</u>			
<u>GM_Circle</u>	<u>gml:Circle</u>	<u>IfcCircle</u>	<u>circl</u> <u>Geom2d_Circle</u> <u>Geom_Circle</u>
		<u>IfcEllipse</u>	<u>elips</u> <u>Geom2d_Ellipse</u> <u>Geom_Ellipse</u>
		<u>IfcLine</u>	<u>line</u> <u>Geom2d_Line</u> <u>Geom_Line</u>
			<u>Geom2d_</u> <u>hyper</u> <u>Hyperbola</u> <u>Geom_Hyperbola</u>
			<u>para</u> <u>Geom2d_Parabola</u> <u>Geom_Parabola</u>
	<u>gml:OffsetCurve</u>	<u>IfcOffsetCurve2D</u>	<u>offset</u> <u>Geom2d_Offset</u> <u>Curve</u> <u>Curve</u> <u>Geom_Offset</u> <u>2d</u> <u>Curve</u>
		<u>IfcOffsetCurve3D</u>	<u>offset</u> <u>Curve</u> <u>3d</u>
<u>IfcOffsetCurveByDistances</u>			
		<u>IfcPcurve</u>	<u>pcur</u>
			<u>bou</u> <u>pcur</u>
<u>IfcPolynomialCurve</u>			
		<u>IfcClothoid</u>	<u>cloth</u>
			<u>circl</u> <u>invo</u>

<u>IfcCosineSpiral</u>	
<u>IfcSecondOrderPolynomialSpiral</u>	
<u>IfcThirdOrderPolynomialSpiral</u>	
<u>IfcSeventhOrderPolynomialSpiral</u>	
<u>IfcSineSpiral</u>	
<u>IfcSurfaceCurve</u>	<u>surf:</u> <u>curv</u>
<u>IfcIntersectionCurve</u>	<u>inter</u> <u>curv</u>
	<u>seam</u> <u>curv</u>
	<u>boundary</u> <u>surf:</u> <u>curv</u>
	<u>complex</u> <u>curv</u> <u>on</u> <u>surf:</u>
	<u>curv</u> <u>repl</u>
<u>IfcSeamCurve</u>	
	<u>Geom2d_Vector</u> <u>WithMagnitude</u> <u>Geom_Vector</u> <u>WithMagnitude</u>
	<u>shapeExtend</u> <u>ComplexCurve</u>

Surface:

Table 20

ISO 19107	GML	IFC	STI OpenCascade	WKT X3D
<u>GM_Surface</u>	<u>gml:Surface</u>			
GM_Polyhedral Surface	<u>gml:PolyhedralSurface</u>			

GM_Triangulated Surface	gml:TriangulatedSurface	IfcTriangulatedFaceSet	tria fac
GM_Tin	gml:Tin		
GM_SurfacePatch	gml:AbstractSurfacePatch		
	gml:Triangle		
	gml:Rectangle		
GM_PointGrid	gml:PointGrid		
GM_Parametric CurveSurface	gml:AbstractParametric CurveSurface		
GM_Gridded Surface	gml:AbstractGridded Surface		
	IfcPlane	pla Geom_Plane	
	IfcCylindricalSurface	cyli Geom_Cylindrical sur Surface	
		cor Geom_Conical sur Surface	
	IfcSphericalSurface	spl Geom_Spherical sur Surface	
	IfcToroidalSurface	tor Geom_Toroidal sur Surface	
		deg tor sur	
		du cyc sur	
	IfcBSplineSurface	b spli Geom_BSpline sur Surface	
	IfcBSplineSurfaceWith Knots	b spli sur wit knc	
		uni sur	

	<u>uas</u>
	<u>uni</u>
	<u>sur</u>
	<u>bez</u> <u>Geom_BezierSurface</u>
	<u>sur</u>
	<u>rati</u>
	<u>b</u>
	<u>spli</u>
	<u>sur</u>
	<u>loc</u>
	<u>refi</u>
	<u>spli</u>
	<u>sur</u>
	<u>rati</u>
	<u>loc</u>
	<u>refi</u>
	<u>spli</u>
	<u>sur</u>
<u>IfcCurveBoundedPlane</u>	<u>cur</u>
<u>IfcCurveBoundedSurface</u>	<u>boi</u>
	<u>sur</u>
	<u>sur</u>
	<u>pat</u>
	<u>off</u> <u>Geom_OffsetSurface</u>
	<u>sur</u>
	<u>ori</u>
	<u>sur</u>
	<u>sur</u>
	<u>rep</u>
	<u>Geom_Surface</u>
	<u>Geom_Osculating</u>
	<u>Surface</u>
<u>IfcRectangularTrimmed</u>	<u>rec</u> <u>Geom_Rectangular</u>
<u>Surface</u>	<u>trin</u> <u>TrimmedSurface</u>
	<u>sur</u>
<u>IfcSectionedSurface</u>	

Primitives:

A primitive (like a block or ellipsoid) can be defined with just a few parameters. It is more efficient than describing the same shape with triangles (mesh) or surfaces (B-rep). It keeps files smaller and mathematically exact.

Table 21

ISO 19107	GML	IFC	STEP	OpenCascade	WKT
		<u>IfcBlock</u>	<u>block</u>	<u>TopoDS_Solid</u> (BRepPrimAPI_MakeBox)	
		<u>IfcRectangularPyramid</u>	<u>rectangular_pyramid</u>		
GM_Cone	<u>gml:Cone</u>	<u>IfcRightCircularCone</u>	<u>right_circular_cone</u> <u>eccentric_cone</u>	<u>TopoDS_Solid</u> (BRepPrimAPI_MakeCone)	
GM_Cylinder	<u>gml:Cylinder</u>	<u>IfcRightCircularCylinder</u>	<u>right_circular_cylinder</u>	<u>TopoDS_Solid</u> (BRepPrimAPI_MakeCylinder)	
			<u>ellipsoid</u>		
				<u>TopoDS_Solid</u> (BRepPrimAPI_MakeHalfSpace)	
				<u>TopoDS_Solid</u> (BRepPrimAPI_MakePrism)	
				<u>TopoDS_Solid</u> (BRepPrimAPI_MakeRevol)	
				<u>TopoDS_Solid</u> (BRepPrimAPI_MakeRevolution)	
GM_Sphere	<u>gml:Sphere</u>	<u>IfcSphere</u>	<u>sphere</u>	<u>TopoDS_Solid</u> (BRepPrimAPI_MakeSphere)	
				<u>TopoDS_Solid</u> (BRepPrimAPI_MakePipe)	
				<u>TopoDS_Solid</u> (BRepPrimAPI_MakePipeShell)	
			<u>torus</u>	<u>TopoDS_Solid</u> (BRepPrimAPI_MakeTorus)	
			<u>right_angular_wedge</u>	<u>TopoDS_Solid</u> (BRepPrimAPI_MakeWedge)	
			<u>tetrahedron</u>		
			<u>convex_hexahedron</u>		
			<u>faceted_primitive</u>		

Boolean Geometry: Defined constructively, not by its boundary. Boolean geometry is the representation of 3D shapes by combining or subtracting simpler solids using Boolean operations like union, intersection, and difference.

Table 22

ISO 19107	GML	IFC	STEP	OpenCascade	WKT
		<u>IfcCsgSolid</u>	<u>Csg_solid</u>		
		<u>IfcBooleanResult</u>	<u>boolean_result</u>	<u>BRepAlgoAPI_BooleanOperation</u>	

Solid/Shell: A solid is a topological object that encloses a region of space and a shell is a set of connected surfaces that form a boundary. A Solid is a closed shell.

Table 23

ISO 19107	GML	IFC	STEP	OpenCascade	WKT
	<u>gml:AbstractSolid</u>				
<u>GM Solid</u>	<u>gml:Solid</u>	<u>IfcFacetedBrep</u>	<u>manifold_solid_brep</u>	<u>TopoDS_Solid (TopoDS_Face)</u>	
		<u>IfcFacetedBrepWithVoids</u>			
		<u>IfcAdvancedBrep</u>	<u>advanced_brep_shape_representation</u>		
		<u>IfcAdvancedBrepWithVoids</u>			
		<u>IfcSectionedSolidHorizontal</u>			
		<u>IfcFixedReferenceSweptAreaSolid</u>			
		<u>IfcDirectrixDerivedReferenceSweptAreaSolid</u>			
		<u>IfcExtrudedAreaSolid</u>	<u>extruded_area_solid</u>		
		<u>IfcExtrudedAreaSolidTapered</u>			
		<u>IfcRevolvedAreaSolid</u>			
		<u>IfcRevolvedAreaSolidTapered</u>			

GM_ Shell	<u>gml:Shell</u>	
		<u>solid_replica</u>
		<u>brep_2d</u>
	<u>IfcSectionedSpine</u>	<u>sectioned_spine</u>
	<u>IfcBoxedHalfSpace</u>	<u>boxed_half_space</u>
	<u>IfcPolygonalBoundedHalfSpace</u>	
		<u>tessellated_solid</u>

Swept Surface:

A swept surface is a type of surface in geometric modeling created by moving a profile (curve) along a trajectory (path).

Table 24

ISO 19107	GML	IFC	STEP	OpenCascade	WKT
		<u>IfcSurfaceCurveSweptArea Solid</u>			
		<u>IfcSweptDiskSolid</u>	<u>swept_disk_solid</u>		
		<u>IfcSweptDiskSolidPolygonal</u>			
			<u>swept_face_solid</u>		
			<u>swept_area_solid</u>		
		<u>IfcSurfaceOfLinearExtrusion</u>	<u>surface_of_linear_ extrusion</u>	<u>Geom_SurfaceOfLinear Extrusion</u>	
		<u>IfcSurfaceOfRevolution</u>	<u>surface_of_revolution</u>	<u>Geom_SurfaceOfRevolution</u>	
		<u>IfcSweptSurface</u>	<u>surface_curve_swept_ surface</u>		
			<u>fixed_reference_swept_ surface</u>		
			<u>revolved_face_solid</u>		

<u>extruded_face_solid</u>
<u>surface_curve_swept_area_solid</u>
<u>swept_disk_solid</u>

Volume:

Volume geometry is a way of representing a 3D object by directly defining the entire region of space it occupies, rather than just its boundary.

Table 25

ISO 19107	GML	IFC	STEP	OpenCascade	WKT	X3D
			<u>block_volume</u>			
			<u>pyramid_volume</u>			
			<u>eccentric_conical_volume</u>			
			<u>spherical_volume</u>			
			<u>ellipsoid_volume</u>			
			<u>cylindrical_volume</u>			
			<u>wedge_volume</u>			
			<u>tetrahedron_volume</u>			
			<u>hexadron_volume</u>			
			<u>toroidal_volume</u>			
			<u>bezier_volume</u>			
			<u>uniform_volume</u>			
			<u>quasi_uniform_volume</u>			
			<u>rational_b_spline_volume</u>			
			<u>trimmed_volume</u>			

<u>locally_refined_spline_volume</u>
<u>rational_locally_refined_spline_volume</u>
<u>b_spline_volume</u>
<u>b_spline_volume_with_knots</u>

Axis/Placement:

Used to define orientation or alignment of objects, where to be placed and in what direction to extrude.

Table 26

ISO 19107	GML	IFC	STEP	OpenCascade	WKT
		<u>IfcAxis1Placement</u>	<u>axis1_placement</u>	<u>Geom_Axis1Placement</u>	
		<u>IfcAxis2Placement2D</u> <u>IfcAxis2Placement3D</u>	<u>axis2_placement_2d</u> <u>axis2_placement_3d</u>	<u>Geom_Axis2Placement</u>	

Grids:

Table 27

ISO 19107	GML	IFC	STEP	OpenCascade	WKT
	<u>gml:RectifiedGrid</u>				
	<u>gml:RectifiedGridCoverage</u>				
	<u>gml:GridCoverage</u>				

Operators:

Table 28

ISO 19107	GML	IFC	STEP	Open Cascade	WKT
			<u>cartesian_transformation_operator</u>		

<u>IfcCartesianTransformationOperator3D</u>	<u>cartesian_</u> <u>transformation_</u> <u>operator_3d</u>	
<u>IfcCartesianTransformationOperator2D</u> <u>IfcCartesianTransformationOperator2Dnon</u> <u>Uniform</u>	<u>cartesian_</u> <u>transformation_</u> <u>operator_2d</u>	<u>gp_Trsf2d</u>

Geometric complex/ composites:

Table 29

ISO 19107	GML	IFC	STEP	Open Cascade	WKT
GM_Complex	<u>gml:Geometric</u> <u>Complex</u>				
<u>GM_Composite</u> <u>Point</u>					
<u>GM_Composite</u> <u>Curve</u>	<u>gml:Composite</u> <u>Curve</u>	<u>IfcComposite</u> <u>Curve</u>	<u>composite_</u> <u>curve</u>	<u>composite_</u> <u>curve</u>	COMPOUNDCURVE (x y (z)(m))
<u>GM_Composite</u> <u>Surface</u>	<u>gml:Composite</u> <u>Surface</u>				
<u>GM_Composite</u> <u>Solid</u>	<u>gml:Composite</u> <u>Solid</u>				
			<u>composite_</u> <u>curve_segment</u>		
			<u>reparametrised_</u> <u>composite_</u> <u>curve_segment</u>		
			<u>rectangular_</u> <u>composite_</u> <u>surface</u>		

Geometric Aggregates:

Table 30

ISO 19107	GML	IFC	STEP	OpenCascade	WKT
<u>GM_MultiPoint</u>	<u>gml:MultiPoint</u>				MULTIPOINT
<u>GM_MultiCurve</u>	<u>gml:MultiCurve</u>				MULTILINESTRING
<u>GM_MultiSurface</u>	<u>gml:MultiSurface</u>				MULTIPOLYGON

<u>GM_MultiSolid</u>	<u>gml:MultiSolid</u>
<u>IfcCartesianPointList2D</u>	
<u>IfcCartesianPointList3D</u>	

Interface Query:

Table 31

ISO 19107	GML	IFC	STEP	OpenCascade	WKT
boundary:Geometry					
convexHull:Geometry					
distance				gp_Circ::Distance() gp_Circ2d::Distance() gp_Lin::Distance(const gp_Pnt &theP) const gp_Lin::Distance(const gp_Lin &theOther) const gp_Lin2d::Distance(const gp_Pnt2d &theP) const gp_Lin2d::Distance(const gp_Lin2d &theOther) gp_Pln::Distance gp_Circ::Distance() etc	
buffer					
intersection				BRepClass_Intersection	
difference					
symDifference					
union					
contains				BRepClass3d_SolidClassifier	
crosses					
disjoint					
equals					
intersects					

overlaps	
touches	
within	
withinDistance	
relate	
3Dboundary:Geometry	
3DconvexHull:Geometry	
3Ddistance	
3Dbuffer	
3Dintersection	BRepClass3d_Intersection3d
3Ddifference	
3DsymDifference	
3Dunion	
3Dcontains	
3Dcrosses	
3Ddisjoint	
3Dequals	
3Dintersects	
3Doverlaps	
3Dtouches	
3Dwithin	
3DwithinDistance	

<https://dev.opencascade.org/doc/refman/html/> <https://old.opencascade.com/doc/occt-7.5.0/overview/html/index.html><https://ifc43-docs.standards.buildingsmart.org/IFC/RELEASE/IFC4x3/HTML/lexical/IfcGeometricRepresentationItem.htm>

<https://www.web3d.org/documents/specifications/19775-1/V4.0/index.html>



17

ANNEX O: HISTORY



BIBLIOGRAPHY





BIBLIOGRAPHY

- [1] Abhayaratna J, van den Brink L, Car N, Atkinson R, Homburg T, Knibbe F, McGlinn K, Wagner A, Bonduel M, Rasmussen MH, Thiery F: Abhayaratna2020wp, *OGC Benefits of Representing Spatial Data Using Semantic and Graph Technologies*. Open Geospatial Consortium (2020). <http://www.opengis.net/doc/wp/using-semantic-graph>.
- [2] Jabi W, Chatzivasileiadi A: Topologic: Exploring Spatial Reasoning Through Geometry, Topology, and Semantics. In: p. 277. Springer International Publishing, Cham. (2021).
- [3] Filip Biljecki HT: QUALITY OF BIM–GIS CONVERSION. (2019).
- [4] Anne Göbels JB: Relative Location Ontology: An ontological model for representing directional topological relationships between spatial entities in oriented space. (2024).
- [5] World Wide Web Consortium: *RDF 1.1 Concepts and Abstract Syntax*, W3C Recommendation, <https://www.w3.org/TR/rdf11-concepts/> (25 February 2014).
- [6] Stotzner E, Homburg T, Mara H: CNN Based Cuneiform Sign Detection Learned from Annotated 3D Renderings and Mapped Photographs with Illumination Augmentation. In: pp. 1680–1688. (2023).
- [7] World Wide Web Consortium: *RDF 1.1 Turtle Terse RDF Triple Language*, W3C Recommendation, <https://www.w3.org/TR/turtle> (25 February 2014).
- [8] Behr J, Eschler P, Jung Y, Zöllner M: X3DOM: a DOM-based HTML5/X3D integration model. In: p. 127. (2009).
- [9] Bonduel M, Wagner A, Pauwels P, Vergauwen M, Klein R: Including widespread geometry formats in semantic graphs using RDF literals. In: vol. 1, p. 341. (2019).
- [10] Brutzman D, Daly L: *X3D: extensible 3D graphics for Web authors*. Elsevier, n.p. (2010).
- [11] Brutzman D, Flotyński J: X3D ontology for querying 3D models on the semantic web. In: p. 1. (2020).
- [12] D'Andrea A, Fernie K: CARARE 2.0: a metadata schema for 3D Cultural Objects. In: vol. 2, p. 137. (2013).
- [13] Donkers S, Ledoux H, Zhao J, Stoter J: Automatic conversion of IFC datasets to geometrically and semantically correct CityGML LOD3 buildings. *Transactions in GIS* vol. 20, p. 547 (2016).
- [14] Feichtenhofer C: X3d: Expanding architectures for efficient video recognition. In: p. 203. (2020).

- [15] Hansson F: Linked geodata: CityGML represented as a virtual knowledge graph. *Student thesis series INES* (2024).
- [16] Kolbe TH, Gröger G, Plümer L: CityGML: Interoperable access to 3D city models. In: p. 883. Springer. (2005).
- [17] Snyderman S, Sanderson R, Cramer T: The International Image Interoperability Framework (IIIF): A community & technology approach for web-based images. In: vol. 12, p. 16. (2015).
- [18] Stötzner E, Homburg T, Bullenkamp JP, Mara H: R-CNN based PolygonalWedge Detection Learned from Annotated 3D Renderings and Mapped Photographs of Open Data Cuneiform Tablets. (2023).
- [19] [NO INFORMATION AVAILABLE]
- [20] [NO INFORMATION AVAILABLE]
- [21] [NO INFORMATION AVAILABLE]
- [22] Chadzynski A, Li S, Griškaitė A, Chua J, Hofmeister M, Yan J, Tai HY, Lloyd E, Tsai YK, Agarwal M, Akroyd J, Herthogs P, Kraft M: Semantic 3D city interfaces—Intelligent interactions on dynamic geospatial knowledge graphs. *Data-Centric Engineering* vol. 4 (2023).
- [23] Perzylo A, Somani N, Rickert M, Knoll A: An ontology for CAD data and geometric constraints as a link between product models and semantic robot task descriptions. In: *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. pp. 4197–4203. IEEE. (2015).
- [24] Flotyński J, Walczak K: Ontology-Based Representation and Modelling of Synthetic 3D Content: A State-of-the-Art Review. *Computer Graphics Forum* vol. 36 no. 8, pp. 329–353 (2017).
- [25] Sikos LF: A novel ontology for 3D semantics: ontology-based 3D model indexing and content-based video retrieval applied to the medical domain. *International Journal of Metadata, Semantics and Ontologies* vol. 12 no. 1, p. 59 (2017).
- [26] Car NJ, Homburg T: GeoSPARQL 1.1: Motivations, Details and Applications of the Decadal Update to the Most Important Geospatial LOD Standard. *ISPRS International Journal of Geo-Information* vol. 11 no. 2, p. 117 (2022).
- [27] El Yamani S, Hajji R, Billen R: IFC-CityGML Data Integration for 3D Property Valuation. *ISPRS International Journal of Geo-Information* vol. 12 no. 9, p. 351 (2023).
- [28] Homburg T, Zwick R, Mara H, Bruhn K: Annotated 3D-Models of Cuneiform Tablets. *Journal of Open Archaeology Data* vol. 10 (2022).
- [29] Haynes R: Evolving Standards in Digital Cultural Heritage – Developing a IIIF 3D Technical Specification. In: *Lecture Notes in Computer Science*. pp. 50–64. Springer International Publishing, Cham. (2023).

- [30] Kutzner T, Chaturvedi K, Kolbe TH: CityGML 3.0: New Functions Open Up New Applications. *PFG – Journal of Photogrammetry, Remote Sensing and Geoinformation Science* vol. 88 no. 1, pp. 43–61 (2020).
- [31] Homburg T, Cramer A, Raddatz L, Mara H: Metadata schema and ontology for capturing and processing of 3D cultural heritage objects. *Heritage Science* vol. 9 no. 1 (2021).
- [32] Ropelewski AJ, Rizzo MA, Swedlow JR, Huisken J, Osten P, Khanjani N, Weiss K, Bakalov V, Engle M, Gridley L, Krzyzanowski M, Madden T, Maiese D, Mandal M, Waterfield J, Williams D, Hamilton CM, Huggins W: Standard metadata for 3D microscopy. *Scientific Data* vol. 9 no. 1 (2022).
- [33] Wang Y, Wang X, Liu A, Zhang J, Zhang J: Ontology of 3D virtual modeling in digital twin: a review, analysis and thinking. *Journal of Intelligent Manufacturing* vol. 36 no. 1, pp. 95–145 (2023).